



Birzeit University

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS5200 - Introduction to Graduation Project

Lumen: A Task-Oriented Assistive System for Blind People

Prepared by:

Diaa Badaha – 1210478

Ahmad Hamdan – 1210241

Omar Husein – 1212738

Section: Section 9

Supervised by: Dr. Mohammad Husein

Date: February 4, 2026

An Introduction Graduation Project submitted to the Department of Electrical and Computer Engineering in partial fulfillment of the requirements for the degree of B.Sc. in Computer Engineering

Abstract

Visual impairment affects millions of individuals worldwide, with indoor navigation remaining a significant challenge due to the limitations of traditional mobility aids and the ineffectiveness of GPS-based systems in indoor environments. Existing assistive vision systems typically employ continuous scene narration, which overwhelms users with constant auditory information and increases cognitive load, or rely on query-based approaches that require users to anticipate their information needs in unfamiliar spaces. This work proposes Lumen, a task-oriented assistive system that addresses this gap through a task-scoped perception paradigm, activating visual processing only in response to explicit user commands.

The proposed system comprises a lightweight wearable device equipped with a camera, microphone, and speaker, connected to cloud-based services that perform object detection and speech recognition. A finite state machine governs system behavior, ensuring predictable task execution and graceful error recovery. Lumen supports two primary task families: Object Allocation, which guides users toward specific items in their environment, and Navigation, which provides landmark-based indoor wayfinding through collaborative waypoint identification. This architecture enables hands-free, voice-controlled interaction while offloading computationally intensive processing to the cloud, allowing the wearable device to remain compact and power-efficient. Several key contributions emerge from this work, most notably the task-scoped interaction paradigm that fundamentally differs from continuous narration approaches by aligning perception activation with user intent. By providing targeted assistance only when explicitly requested, Lumen aims to enhance independent navigation for blind and visually impaired individuals while respecting user autonomy and reducing cognitive burden.

المستخلص

يؤثر ضعف البصر على ملايين الأفراد حول العالم، ولا تزال الملاحة داخل المباني تمثل تحدياً كبيراً بسبب محدودية وسائل التنقل التقليدية وعدم فعالية أنظمة تحديد المواقع المعتمدة على نظام تحديد المواقع العالمي GPS في البيئات الداخلية. تعتمد أنظمة الرؤية المساندة الحالية غالباً على السرد المستمر للمشهد، مما يرهق المستخدم بتدفق دائم من المعلومات السمعية ويزيد العبء المعرفي، أو على أساليب قائمة على الاستعلام تتطلب من المستخدم توقع احتياجاته المعلوماتية مسبقاً في أماكن غير مألوفة. يقدم هذا العمل نظام Lumen، وهو نظام مساعد موجه بالمهام يعالج هذه الفجوة من خلال تبني نموذج مساعد يعمل بناءً على مهام معينة، حيث يتم تفعيل المعالجة البصرية فقط استجابةً لأوامر صريحة من المستخدم. يتكون النظام المقترح من جهاز قابل للارتداء وخفيف الوزن مزود بكاميرا وميكروفون وسماعات، ومتصل بخدمات سحابية تتولى مهام كشف الأجسام والتعرف على الكلام. تتحكم آلة حالات منتهية في سلوك النظام، بما يضمن تنفيذاً متوقعاً للمهام وآليات تعافٍ سلسلة من الأخطاء. يدعم Lumen فئتين رئيسيتين من المهام: تحديد الأجسام، التي ترشد المستخدمين إلى عناصر محددة في البيئة المحيطة، والملاحة (التحرك)، التي توفر إرشاداً داخلياً قائماً على المعالم من خلال تحديد نقاط طريق تعاونية. تتيح هذه البنية تفاعلاً بدون استخدام اليدين وبالاعتماد الكامل على الأوامر الصوتية، مع ترحيل المعالجة الحسابية المكثفة إلى السحابة، مما يسمح بالحفاظ على جهاز قابل للارتداء صغير الحجم وموفر للطاقة. تنبثق عن هذا العمل عدة إسهامات أساسية، أبرزها نموذج التفاعل المقيّد بالمهام الذي يختلف جوهرياً عن أساليب السرد المستمر من خلال مواءمة تفعيل الإدراك مع نية المستخدم. ومن خلال تقديم مساعدة موجهة فقط عند الطلب الصريح، يهدف Lumen إلى تعزيز الاستقلالية في التنقل لدى المكفوفين وضعاف البصر مع احترام استقلالية المستخدم وتقليل العبء المعرفي.

Contents

English Abstract	I
Arabic Abstract	II
Table of Contents	III
List of Figures	VI
List of Tables	VII
List of Abbreviations	VIII
1 Introduction and Motivation	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 High-Level Methodology	3
1.4 Contributions	4
1.5 Report Outline	4
2 Background	6
2.1 Vision-Based Perception for Assistive Systems	6
2.1.1 Object Detection	6
2.1.2 Relative Spatial Reasoning	7
2.1.3 Obstacle Detection and Environmental Awareness	7
2.1.4 Landmark-Based Perception	7
2.2 Voice-Based Human–System Interaction	8
2.2.1 Speech Recognition	8
2.2.2 Language Understanding	8
2.2.3 Speech Synthesis	8
2.3 Task-Oriented Assistive Interaction	9
2.3.1 Task-Based Interaction Paradigms	9
2.3.2 State-Based System Control	9

2.3.3	Cognitive Load in Assistive Interaction	9
3	Related Work	10
3.1	Assistive Vision Systems for Blind Users	10
3.2	Interaction Paradigms: Continuous Narration vs On-Demand Assistance	11
3.3	Indoor Navigation and Object-Finding Systems	12
3.4	Cognitive Load and Information Management in Assistive Technologies .	13
3.5	Wearable Platforms and Voice-Based Interaction	14
3.6	System Architectures, Control Models, and Safety in Assistive Systems .	15
4	Methodology	16
4.1	System Overview	16
4.1.1	System Architecture	16
4.1.2	Component Overview	17
4.1.3	Data Flow and Interaction	18
4.1.4	Design Rationale	18
4.2	Hardware Setup	20
4.2.1	Hardware Components	20
4.2.2	Physical Design, Integration, and Mounting	21
4.2.3	System Cost	23
4.2.4	Design Rationale and Alternatives	23
4.3	Software Architecture	23
4.3.1	Architecture Overview	23
4.3.2	Device-Side Software	24
4.3.3	Cloud Server Software	25
4.3.4	Data Flow and Communication Protocol	29
4.4	Task Definition and Execution Pipeline	29
4.4.1	Task Family Overview	29
4.4.2	Object Allocation Task Execution	30
4.4.3	Navigation Task Execution	37
4.4.4	Task Execution Examples	41
4.5	Finite State Machine Design	43
4.5.1	FSM States	44
4.5.2	State Transitions	44
4.5.3	State Transitions	45
4.6	Error Handling and Recovery	45
4.6.1	Error Categories and Recovery Strategies	45
4.6.2	Graceful Degradation	46

5	Evaluation	47
5.1	Experimental Setup	47
5.2	Perception and Spatial Guidance Performance	48
5.3	Voice Interaction Performance	48
5.4	End-to-End Task Performance	48
5.5	FSM Behavior and Safety Evaluation	49
5.6	Qualitative Observations and Limitations	49
6	Conclusion and Future Work	50
6.1	Summary of Contributions	50
6.2	Limitations	51
6.2.1	Technical Limitations	51
6.2.2	Design Limitations	52
6.3	Future Work	52
6.3.1	System Implementation and Prototyping	52
6.3.2	Dataset Collection and Model Refinement	53
6.3.3	Performance Optimization	53
6.3.4	Feature Enhancement	53
6.3.5	Privacy and Security Considerations	53
6.3.6	Implementation Timeline	53
6.3.7	Toward Practical Assistive Technology	54
	References	55

List of Figures

2.1	Example of Object Detection.[11].	6
2.2	Overview of a Speech Recognition Pipeline[14].	8
4.1	Hardware Component Diagram	22
4.2	Full Mounted Device Illustration	22
4.3	Software Architecture Diagram	24
4.4	Object Allocation Task Flowchart	31
4.5	Object Allocation Task Sequence Diagram	32
4.6	Navigation Task Flowchart	37
4.7	Navigation Task Sequence Diagram	38
4.8	System Finite State Machine Diagram	43
6.1	Implementation Timeline and Work Breakdown	54

List of Tables

4.1	Hardware Component Specifications	20
4.2	COCO Dataset Classes Relevant to Indoor Navigation	26
4.3	FSM State Transition Rules	44

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
ARM	Advanced RISC Machine
COCO	Common Objects in Context
CSI	Camera Serial Interface
FSM	Finite State Machine
GHz	Gigahertz
GPS	Global Positioning System
gTTS	Google Text-to-Speech
HCI	Human-Computer Interaction
I/O	Input/Output
JSON	JavaScript Object Notation
LLM	Large Language Model
MB	Megabyte
MP	Megapixel
NLG	Natural Language Generation
RAM	Random Access Memory
STT	Speech-to-Text
TTS	Text-to-Speech
USB	Universal Serial Bus
WiFi	Wireless Fidelity
YOLO	You Only Look Once
YOLOv8	You Only Look Once Version 8
YOLOv8n	YOLOv8 Nano
YOLOv8s	YOLOv8 Small

Chapter 1

Introduction and Motivation

1.1 Motivation

Visual impairment affects approximately 2.2 billion people worldwide, including 39 million who are completely blind [1]. These numbers continue rising due to population growth and aging, making independent navigation a critical challenge for millions of individuals.

Among the most pressing difficulties is indoor navigation, which remains largely unsolved despite significant advances in outdoor GPS-based systems [2]. The inability to navigate independently in unfamiliar indoor spaces—offices, hospitals, shopping centers—severely limits employment opportunities, social participation, and overall quality of life for blind individuals.

Traditional mobility aids have inherent limitations. The white cane detects obstacles only within physical reach, missing overhead hazards and providing no advance warning [3]. Guide dogs, while effective, require extensive training, cost between \$15,000 and \$50,000, and are not suitable for all users due to various constraints [4]. Indoor environments further complicate navigation as building structures attenuate GPS signals, rendering them unreliable [5].

Recent electronic travel aids have attempted to address these challenges through continuous scene narration—constantly verbalizing environmental information. However, this approach introduces a critical problem: cognitive overload. Users must process continuous auditory streams while simultaneously attending to ambient sounds, tactile feedback, and spatial reasoning [6]. Research shows this can be overwhelming and counterproductive, causing users to miss critical details.

This tension between providing sufficient navigation information and avoiding cognitive overload motivates a different approach. Rather than continuous environmental description, systems could activate perception only when users explicitly request assistance for specific tasks. This task-oriented paradigm could reduce cognitive load, improve user control, and prove more practical for real-world navigation.

1.2 Problem Statement

Current assistive navigation systems for blind users typically fall into two categories, each with fundamental limitations. The first category comprises continuous narration systems that constantly describe the surrounding environment through audio feedback. These systems attempt to provide comprehensive scene understanding by continuously verbalizing detected objects, spatial relationships, and environmental changes. While this approach appears thorough, it creates severe cognitive load problems. Users must simultaneously process continuous auditory information while attending to ambient sounds, tactile feedback from their mobility aids, and spatial reasoning [7]. Research on cognitive load in assistive technologies has demonstrated that this constant information flow can overwhelm users, leading to reduced task performance and increased mental fatigue.

The second category consists of query-based systems where users must explicitly request information about specific aspects of their environment [5], [8]. While these systems reduce cognitive load by providing information only on demand, they introduce a different problem: users must know precisely what to ask for and when. In unfamiliar environments, blind users often cannot anticipate what information they need until it becomes critically necessary. This creates a reactive rather than proactive assistance paradigm, where users may miss important environmental cues simply because they did not know to query for them.

Both approaches share a common limitation: they treat perception as either continuously active or completely passive. Continuous systems waste computational resources and overwhelm users with unnecessary information during periods when no specific task requires guidance. Query-based systems place the burden entirely on users to manage their perception needs, requiring constant situational awareness of what information might be relevant. Neither approach aligns well with how blind individuals naturally navigate—performing specific tasks such as locating objects, reaching destinations, or avoiding obstacles, each with distinct information requirements.

What remains missing is a middle ground: a system that activates perception and provides guidance in response to explicit user-initiated tasks, but does so intelligently without requiring users to specify every detail. Such a system would combine the proactive assistance of continuous narration with the cognitive efficiency of on-demand querying, while avoiding the pitfalls of both. The core research challenge is designing an interaction paradigm and system architecture that enables task-scoped perception—where visual processing and guidance activate only during explicitly requested tasks, provide structured assistance toward task completion, and return to an idle state once the task concludes. This approach could address the research question: *How can assistive navigation systems provide reliable, context-aware guidance while minimizing cognitive load through task-oriented perception activation?*

1.3 High-Level Methodology

To address the gap identified above, Lumen introduces a task-oriented assistive vision system that activates perception only in response to explicit user commands. The system comprises a compact wearable device with an integrated camera, microphone, WiFi connectivity, and speaker, enabling hands-free operation.

Lumen employs cloud-based processing for computer vision and speech recognition, leveraging powerful artificial Intelligence (AI) models without local computational constraints. When a user issues a voice command, the camera streams video to cloud servers where object detection and scene understanding occur in real-time. Processed results are converted to audio guidance and transmitted back to the device’s integrated speaker. This architecture enables state-of-the-art deep learning models that would be computationally infeasible on embedded hardware, ensuring high accuracy in object recognition and spatial understanding.

The system organizes assistance into two primary task families:

- **Object Allocation:** Helps users locate specific items in their environment, such as personal belongings or room features. The system provides spatial guidance directing users toward the detected object’s location.
- **Navigation:** Provides route guidance for reaching destinations such as rooms, doorways, or specific landmarks within indoor spaces. The system offers turn-by-turn directional instructions to guide users safely to their intended location.

A finite state machine (FSM) manages the system’s operational states, ensuring perception activates only during explicit user tasks. The FSM transitions between idle and active states based on voice commands, maintaining the system in a low-power idle state when not actively assisting. This state-based control eliminates the cognitive burden of continuous environmental narration while avoiding the limitations of passive query systems that require users to anticipate their information needs.

Users interact with Lumen entirely through voice commands to initiate tasks and receive audio guidance through the wearable device’s integrated speaker. The interaction flow follows a natural pattern: the user speaks a command such as “find my cup” or “navigate to the kitchen,” the system processes the request and activates the appropriate task, provides real-time audio guidance, and returns to idle state upon task completion or user cancellation. The wireless connectivity enables seamless communication with cloud processing infrastructure, while the wearable form factor ensures the system remains accessible and unobtrusive during daily activities. This task-scoped architecture addresses the fundamental challenge of balancing comprehensive assistance with cognitive efficiency, providing guidance precisely when needed without overwhelming users with unnecessary information.

1.4 Contributions

The main contributions of this project are:

- **Task-Scoped Perception Paradigm:** A task-oriented perception paradigm that activates vision processing only during explicit user commands, eliminating continuous cognitive load while maintaining proactive assistance.
- **Cognitive Load Reduction Framework:** A framework for minimizing cognitive load in assistive systems through FSM-based state management and event-driven feedback.
- **Cloud-Native Assistive Architecture:** A cloud-based architecture enabling resource-constrained wearable devices to leverage state-of-the-art computer vision models without local processing overhead.
- **Integrated Wearable Device with Hands-Free Interaction:** An integrated wearable design combining camera, audio I/O, and wireless connectivity with completely hands-free voice-controlled operation, requiring no manual input and maximizing accessibility for blind users.
- **Real-Time Context-Aware Guidance Pipeline:** A low-latency video streaming and audio feedback pipeline that provides spatially-aware guidance adapted to task context and user location in real-time.

1.5 Report Outline

This chapter has presented the motivation, problem statement, high-level methodology, and contributions of this work. The remainder of this report is organized as follows:

- **Chapter 2: Background** provides the foundational concepts necessary to understand the system design, including assistive vision systems, voice-based interaction, object detection fundamentals, task-based system design, and finite state machines.
- **Chapter 3: Related Work** surveys existing assistive navigation systems, scene description approaches, smart glasses and wearable devices, and voice assistants for blind users, positioning Lumen within the current research landscape.
- **Chapter 4: Methodology** presents the detailed system design, including hardware setup, software architecture, task definitions and execution pipeline, FSM design, interaction flow, and error handling mechanisms.

- **Chapter 5: Evaluation** describes the experimental setup, test scenarios, evaluation metrics, quantitative results, failure case analysis, and discussion of system performance.
- **Chapter 6: Conclusion and Future Work** summarizes the contributions of this work, discusses current limitations, and outlines directions for future research and system enhancements.

Chapter 2

Background

2.1 Vision-Based Perception for Assistive Systems

2.1.1 Object Detection

Object detection is a fundamental task in computer vision that aims to identify objects of interest within an image while also estimating their spatial location. Unlike image classification, which assigns a single label to an entire image, object detection enables the recognition and localization of multiple objects within complex scenes, a capability that is essential for environment-aware systems [9]. As illustrated in Figure 2.1, detected objects are typically represented using bounding boxes associated with semantic class labels, allowing both identification and spatial interpretation of the scene. Recent advances in real-time object detection architectures, such as single-stage detectors, have further enabled the deployment of vision-based perception in interactive and assistive contexts where timely scene understanding is required [10].



Figure 2.1: Example of Object Detection.[11].

2.1.2 Relative Spatial Reasoning

Relative spatial reasoning refers to the qualitative interpretation of object locations with respect to an observer using descriptors such as left, right, front, or near. In assistive applications, such qualitative spatial representations are often preferred over precise metric coordinates, as they better align with human perception and natural language communication [3]. Studies on assistive navigation technologies indicate that relative spatial cues can support intuitive guidance and reduce cognitive load, particularly in indoor environments where exact localization is unnecessary [2].

2.1.3 Obstacle Detection and Environmental Awareness

Obstacle detection refers to the identification of objects or structures that may impede safe movement within an environment. In assistive contexts, obstacle awareness is a critical component of navigation support, as visually impaired users rely on timely warnings to avoid collisions and hazards. Vision-based obstacle detection approaches have been widely explored as a means to enhance environmental awareness, enabling systems to recognize common indoor obstacles such as furniture and other navigational hazards [3]. Studies on assistive navigation technologies further indicate that obstacle detection plays a key role in improving user safety and confidence, particularly in unfamiliar or cluttered environments [2].

2.1.4 Landmark-Based Perception

Landmark-based perception refers to the use of salient objects or environmental structures as reference points for spatial orientation and navigation. In indoor assistive contexts, landmarks such as doors, furniture, or distinctive objects provide meaningful cues that help users understand their position and movement within an environment. Unlike precise metric localization, landmark-based representations rely on recognizable and stable features that align with human spatial cognition and natural descriptions of space [2]. Prior research on assistive navigation systems has shown that leveraging landmarks can improve wayfinding effectiveness and reduce cognitive load, particularly in unfamiliar indoor environments where maps or global positioning information may be unavailable or unreliable [3].

2.2 Voice-Based Human–System Interaction

2.2.1 Speech Recognition

Speech recognition is the process of converting spoken language into textual representations that can be processed by computational systems. Modern SR approaches rely on statistical and neural acoustic modeling techniques to handle variability in pronunciation, speaking rate, and environmental noise [12]. As illustrated in Figure 2.2, speech recognition systems typically follow a structured processing pipeline that transforms raw audio signals into linguistic units before producing textual output. As a result, speech recognition has become a central component of voice-driven interfaces, enabling hands-free interaction and natural command-based control in assistive and interactive systems [13].

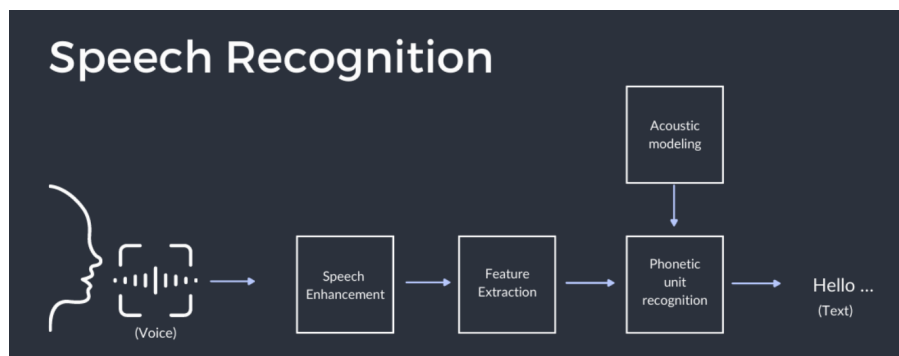


Figure 2.2: Overview of a Speech Recognition Pipeline[14].

2.2.2 Language Understanding

Language understanding focuses on extracting semantic meaning from textual input and mapping it to structured representations such as intents, actions, and parameters. In task-oriented interaction, LU is primarily concerned with identifying user goals from short and often imperative utterances rather than full conversational dialogue [15]. This capability allows systems to reason about user intent beyond literal transcription and to support goal-directed interaction within constrained domains [16].

2.2.3 Speech Synthesis

Speech synthesis is the process of generating spoken audio from textual input. Traditional speech synthesis techniques relied on concatenative or parametric methods, while more recent neural approaches have significantly improved the naturalness and intelligibility of synthesized speech [17]. In voice-based interaction scenarios, speech synthesis serves as a primary output modality, enabling systems to convey information and guidance through auditory feedback when visual channels are unavailable or unsuitable [18].

2.3 Task-Oriented Assistive Interaction

2.3.1 Task-Based Interaction Paradigms

Task-based interaction paradigms organize system behavior around explicit user-initiated tasks rather than continuous or passive operation. In this paradigm, perception and processing are activated in response to a user request and remain scoped to the duration of the task, which helps limit unnecessary system output [19]. Research on assistive navigation and interaction systems has shown that task-based approaches can improve usability and reduce cognitive load by focusing system responses on the user’s immediate goal [2].

2.3.2 State-Based System Control

State-based system control models system behavior as a finite set of operational states with well-defined transitions triggered by events or conditions. This approach provides a structured framework for managing complex interactions, handling interruptions, and enforcing predictable behavior [20]. State-based control mechanisms are widely adopted in assistive and safety-critical systems, where clear task boundaries and robust recovery behavior are required [21].

2.3.3 Cognitive Load in Assistive Interaction

Cognitive load refers to the mental effort required to process information and make decisions during interaction. In assistive systems, managing cognitive load is especially important, as excessive or poorly timed feedback can overwhelm users and reduce usability. Research in human–computer interaction emphasizes that assistive technologies should prioritize concise, task-relevant information delivery to minimize cognitive burden [20]. Prior work on assistive navigation systems has shown that structured interaction and limited feedback frequency can improve user performance and reduce confusion during navigation and object localization tasks [19].

Chapter 3

Related Work

3.1 Assistive Vision Systems for Blind Users

Assistive vision systems for blind and visually impaired users have been an active area of research for several decades, driven by advances in computer vision, wearable sensing, and audio-based interaction. These systems typically employ cameras mounted on wearable or handheld devices to capture visual information from the user’s environment and translate it into non-visual feedback, most commonly through speech or spatialized audio. The primary objective of such systems is to enhance environmental awareness, enabling users to perceive obstacles, objects, and spatial layout that would otherwise be inaccessible through traditional mobility aids alone.

Early assistive vision approaches focused on specific perceptual functions such as obstacle detection, depth estimation, and object recognition, often providing simple auditory cues or alerts to indicate the presence of hazards [3]. As computer vision techniques matured, more comprehensive systems emerged that attempted to describe broader aspects of the surrounding scene, including object categories, relative positions, and semantic context [5]. These systems generally rely on continuous or near-continuous visual processing pipelines, transforming camera input into spoken descriptions intended to substitute or augment visual perception.

More recent work has explored increasingly sophisticated assistive vision capabilities, including semantic scene understanding, indoor layout recognition, and landmark identification [2]. Advances in deep learning have enabled higher recognition accuracy and robustness in complex indoor environments, while improvements in wearable hardware have facilitated hands-free operation through smart glasses and body-mounted cameras. Many of these systems emphasize real-time feedback and aim to provide a rich representation of the user’s surroundings, often prioritizing completeness of information over selectivity.

Despite significant progress, existing assistive vision systems vary widely in how per-

ception is activated, how information is structured, and how feedback is delivered to users. While some systems prioritize continuous environmental awareness, others restrict assistance to specific perceptual functions or interaction modes. This diversity reflects differing assumptions about user needs and system goals, and it motivates a closer examination of interaction paradigms, information delivery strategies, and control structures in assistive technologies, which are discussed in the following sections.

3.2 Interaction Paradigms: Continuous Narration vs On-Demand Assistance

A central design dimension in assistive vision systems is the interaction paradigm that governs when perception is activated and how information is delivered to users. Prior work largely falls into two dominant paradigms: continuous scene narration and on-demand, query-based assistance. These paradigms differ fundamentally in how they balance environmental awareness, user control, and cognitive load.

Continuous narration systems aim to provide persistent situational awareness by continuously processing visual input and verbalizing detected objects, obstacles, and scene elements [2], [5]. By maintaining an always-on perception pipeline, these systems attempt to approximate visual awareness through ongoing auditory descriptions. While this approach can increase environmental coverage, it often results in verbose and redundant feedback, requiring users to continuously filter relevant information from irrelevant details. Studies in assistive technology and auditory interfaces have shown that such uninterrupted information streams can significantly increase cognitive load, interfere with attention to ambient sounds, and reduce overall task performance [6], [7].

In contrast, on-demand and query-based assistive systems restrict perception and feedback to moments explicitly initiated by the user through individual queries [5], [8]. These systems typically rely on voice commands or manual input to request specific information, such as identifying nearby objects or describing the scene ahead. By limiting feedback to user-triggered queries, on-demand approaches reduce continuous cognitive burden and offer greater control over information flow. However, they introduce a different challenge: users must know what to ask for and when. In unfamiliar environments, blind users may lack sufficient situational context to formulate appropriate queries, leading to delayed or incomplete assistance.

Both paradigms reflect implicit assumptions about user behavior and system responsibility. Continuous narration systems assume that more information leads to better awareness, while query-based systems assume that users can effectively manage their own information needs. In practice, neither assumption consistently holds across real-world navigation tasks. Continuous systems risk overwhelming users with unnecessary detail,

whereas purely reactive systems shift the burden of situational assessment entirely onto the user. As a result, existing interaction paradigms struggle to provide timely, relevant assistance without either excessive verbosity or excessive user effort.

This tension highlights a gap in current assistive vision research: the lack of interaction models that dynamically align perception and feedback with user intent and task context. Rather than treating perception as either continuously active or fully passive, prior work suggests the need for interaction paradigms that structure assistance around meaningful user activities, an issue further explored in subsequent sections.

3.3 Indoor Navigation and Object-Finding Systems

Indoor navigation and object-finding represent two of the most common and challenging assistive tasks for blind and visually impaired users. Unlike outdoor navigation, which can rely on GPS and large-scale mapping, indoor environments are highly structured, dynamic, and GPS-denied, requiring alternative sensing and localization strategies [2]. As a result, a wide range of assistive systems have been proposed to support indoor wayfinding and target localization through vision-based, infrastructure-assisted, or hybrid approaches.

Indoor navigation systems for blind users often focus on guiding users between pre-defined locations using landmarks, visual markers, or spatial cues extracted from the environment [22], [23]. Vision-based approaches leverage cameras to detect doors, corridors, signage, and other structural elements, translating this information into turn-by-turn audio instructions. While these systems can provide effective guidance in controlled settings, many rely on continuous perception pipelines and assume persistent system engagement throughout the navigation task. Additionally, some approaches require prior mapping, fiducial markers, or dedicated infrastructure, limiting scalability and deployment flexibility.

Object-finding systems, in contrast, are typically designed to help users locate specific items within their immediate surroundings, such as personal belongings or household objects [24], [25]. These systems commonly employ real-time object detection and spatial audio cues to guide users toward a target. Although object-finding is often treated as a simpler problem than navigation, it presents similar challenges in perception reliability, spatial feedback, and interaction flow. Many existing systems continuously scan the environment or require repeated user queries, offering limited structure for task initiation, progression, and termination.

Across both navigation and object-finding domains, a common pattern emerges: systems are primarily designed around continuous sensing or fixed interaction loops, rather than explicit task boundaries. Navigation assistance is frequently modeled as a long-running process, while object-finding systems often lack clear mechanisms for transition-

ing between idle, active, and completed states. As a result, perception and feedback remain active even when no task-relevant information is required, potentially increasing cognitive load and reducing usability during extended use.

These observations suggest that while substantial progress has been made in enabling indoor navigation and object localization for blind users, prior work often treats these capabilities as standalone functions rather than structured, goal-driven activities. This motivates further examination of how assistive systems can better align perception, guidance, and interaction flow with discrete user tasks and intentions.

3.4 Cognitive Load and Information Management in Assistive Technologies

Cognitive load has been widely recognized as a critical factor influencing the effectiveness and usability of assistive technologies for blind and visually impaired users. Assistive systems frequently rely on auditory feedback as the primary communication channel, requiring users to process spoken information while simultaneously attending to environmental sounds, spatial cues, and physical navigation tasks. Prior research has shown that excessive or poorly structured auditory information can overwhelm users, leading to reduced task performance, increased mental fatigue, and diminished situational awareness [6], [7].

Several studies have examined how continuous auditory feedback affects attention and information processing in assistive contexts. Continuous narration systems, in particular, tend to produce high volumes of verbal output, often including redundant or low-priority information [5]. While such systems aim to maximize environmental awareness, they can inadvertently increase cognitive burden by forcing users to continuously filter relevant cues from irrelevant descriptions. This effect is amplified in indoor environments, where navigation requires constant spatial reasoning and sensitivity to ambient sounds such as footsteps, echoes, and human activity [3].

Beyond the quantity of information, prior work emphasizes the importance of timing, structure, and context in information delivery. Research in auditory interfaces and assistive interaction design suggests that information presented outside of an immediate task context is less likely to be retained and more likely to interfere with concurrent cognitive processes [26]. Systems that fail to align feedback with user intent risk either distracting users during critical moments or providing guidance too late to be actionable.

Despite these insights, many assistive vision systems treat cognitive load as a secondary concern, addressing it primarily through ad hoc reductions in verbosity or simplified output formats. Less attention has been given to architectural mechanisms that explicitly regulate when perception is active and when information is delivered. As a

result, cognitive load management is often handled implicitly rather than through structured system design.

These findings suggest that effective assistive systems must manage information flow at a systemic level, controlling not only the content of feedback but also its activation, duration, and relevance to the user’s current task. This perspective motivates the exploration of interaction and control strategies that treat cognitive load management as a first-class design objective rather than an afterthought.

3.5 Wearable Platforms and Voice-Based Interaction

Wearable platforms have become a dominant form factor for assistive technologies targeting blind and visually impaired users, as they enable potentially continuous sensing while preserving hands-free operation. Prior work has explored a range of wearable designs, including smart glasses, head-mounted cameras, chest-mounted devices, and belt- or shoulder-worn systems [3], [25]. These platforms typically integrate a camera for visual sensing and audio output for feedback, aiming to augment environmental awareness without interfering with traditional mobility aids such as the white cane.

The design of wearable assistive systems is constrained by several practical factors, including device size, weight, battery life, and computational resources. As a result, many systems rely on external processing, either through paired smartphones or cloud-based services, to perform computationally intensive vision tasks [27]. Cloud-assisted approaches enable the use of advanced computer vision models and scalable processing pipelines, but they also introduce challenges related to latency, connectivity, and real-time interaction.

Voice-based interaction has emerged as a natural and widely adopted modality for controlling wearable assistive systems. Speech input allows users to initiate actions without manual controls, while audio output provides an accessible channel for conveying guidance and system feedback [19]. Prior work has leveraged voice commands for tasks such as scene description, object identification, and navigation queries. However, while speech interfaces simplify interaction, they also require careful design to avoid ambiguity, misrecognition, and excessive dialogue overhead, particularly in noisy indoor environments.

Despite advances in wearable sensing and voice interfaces, many assistive systems treat interaction as a thin control layer rather than a structured process. Voice commands are often mapped directly to isolated functions, with limited consideration of interaction flow, task progression, or system state. As a result, users may need to repeatedly issue commands or manage system behavior implicitly, increasing interaction effort and cognitive demand. This observation suggests that effective wearable assistive platforms require not only appropriate hardware and input modalities, but also well-defined interaction structures that govern how tasks are initiated, executed, and terminated.

3.6 System Architectures, Control Models, and Safety in Assistive Systems

The effectiveness of assistive technologies depends not only on sensing accuracy and interaction modalities, but also on the underlying system architecture that governs task execution, control flow, and error handling. Prior work in assistive vision and navigation systems commonly describes functional components such as perception, localization, and feedback generation; however, the control logic that coordinates these components is often left implicit or loosely defined [3], [25]. As a result, system behavior is frequently specified at a high level without formal mechanisms for managing task progression, interruptions, or recovery.

Many assistive systems adopt event-driven or reactive architectures, where sensory inputs directly trigger responses such as audio alerts or spoken descriptions [19]. While such designs can simplify implementation, they often lack explicit representations of system state, making it difficult to reason about behavior over time. In the absence of structured control models, perception may remain active indefinitely, feedback may be repeated unnecessarily, and system responses may vary unpredictably depending on sensor noise or user input timing.

Safety and failure handling represent particularly critical yet underexplored aspects of assistive system design. For blind users, incorrect guidance, delayed feedback, or ambiguous instructions can have immediate physical consequences. Despite this, many assistive systems rely on ad hoc safeguards, such as confidence thresholds or simple error messages, rather than explicit safety states or recovery strategies [2]. Few systems formally specify how to halt assistance when confidence is low, how to recover from misdetections, or how to transition cleanly between operational modes in response to user intervention.

In related fields such as robotics and human–computer interaction, state-based control models, including finite state machines and hierarchical task representations, have been widely used to provide deterministic behavior, clear task boundaries, and predictable recovery mechanisms [28], [29]. However, these approaches have seen limited adoption in assistive vision systems for blind users, where control logic is often embedded implicitly within application code or learned models, rather than expressed as explicit system structure.

The lack of formalized control models in existing assistive systems complicates both usability and evaluation, as system behavior under edge cases and failure conditions is difficult to predict or reproduce. These observations suggest a need for assistive system architectures that explicitly encode task structure, control flow, and safety considerations, enabling more reliable, interpretable, and user-centered behavior.

Chapter 4

Methodology

4.1 System Overview

Lumen is proposed as a cloud-assisted assistive navigation system that combines a wearable sensing device with cloud-based perception and reasoning services. The system is designed to support task-oriented interaction for blind and visually impaired users, activating visual perception and guidance only in response to explicit user requests. This section presents the intended high-level system architecture, outlines the major components and their roles, and motivates key design choices underlying the proposed approach.

4.1.1 System Architecture

The proposed system architecture follows a distributed processing model in which computationally intensive perception and reasoning tasks are performed in the cloud, while the wearable device is responsible for sensing, audio interaction, and basic control. This architectural choice is motivated by the need to balance computational capability, device size, power consumption, and cost in a wearable assistive system.

At a high level, the system is structured into three conceptual layers:

- **Client Layer:** A wearable device intended to capture visual data and user voice input, transmit task-relevant information to the cloud, and deliver audio guidance back to the user.
- **Transport Layer:** A wireless communication layer that supports bidirectional data exchange between the wearable device and cloud services. This layer is designed to abstract communication details from higher-level system logic.
- **Application Layer:** Cloud-based services responsible for speech recognition, visual perception, task management, and guidance generation.

This layered structure is intended to separate concerns between sensing, communication, and intelligent processing, enabling modular development and future system evolution. By decoupling the wearable device from computationally heavy processing, the architecture allows perception models, interaction logic, and reasoning components to be updated or replaced without requiring changes to the physical device.

The cloud services are envisioned to expose a well-defined application interface that the wearable client can invoke during active tasks. This loose coupling supports extensibility and experimentation with alternative models or interaction strategies, while preserving the task-scoped perception paradigm central to the system design.

4.1.2 Component Overview

The proposed system is conceptually decomposed into a set of functional components distributed across the wearable device and cloud infrastructure.

Client Layer Components

The wearable device is intended to host a minimal set of components responsible for sensing, user interaction, and communication with cloud services:

1. **Camera Module:** Intended to capture visual observations of the user’s surroundings during active tasks and provide input for cloud-based perception.
2. **Microphone Module:** Intended to capture user voice commands for speech recognition and task initiation.
3. **Speaker Module:** Intended to deliver synthesized audio guidance and system feedback to the user.
4. **Wireless Communication Interface:** Intended to manage bidirectional data exchange between the wearable device and cloud services during active tasks.

Application Layer Components

The cloud infrastructure is intended to host the system’s computationally intensive processing and task coordination logic:

5. **Video Receiver Service:** Intended to accept visual data from the wearable device and route it to perception modules.
6. **Object Detection Module:** Intended to analyze incoming visual observations to identify objects, obstacles, and navigation-relevant landmarks.

7. **Speech Recognition Module:** Intended to convert user voice input into text and extract task intent.
8. **Task Manager and FSM Controller:** Intended to coordinate task execution, manage system state transitions, and determine appropriate guidance behavior.
9. **Audio Synthesis Module:** Intended to generate spoken guidance and feedback based on task context.

4.1.3 Data Flow and Interaction

The proposed system follows an event-driven interaction model centered on explicit user commands rather than continuous sensing. When idle, the wearable device remains in a low-activity state while listening for voice input, and cloud services remain available to respond to task requests. System interaction is organized around discrete task activations rather than fixed message sequences or continuous data streams.

During an active task, interaction follows a high-level progression:

- The user initiates a task through a voice command.
- The wearable device captures and transmits task-relevant sensory data.
- Cloud services interpret the command and activate appropriate perception and reasoning modules.
- Task-specific guidance is generated and returned to the device as audio feedback.
- The system returns to an idle state upon task completion or cancellation.

This interaction design ensures that perception and processing are activated only when explicitly required by the user’s intent. By structuring data flow around task boundaries rather than continuous streams, the system aims to reduce unnecessary computation, network usage, and cognitive load while maintaining responsive and context-aware assistance.

4.1.4 Design Rationale

This subsection summarizes the key design choices underlying the proposed system. These decisions reflect trade-offs considered during the design phase and are motivated by accessibility, cognitive efficiency, and practical deployment constraints.

Cloud-Based Processing

The system is designed to rely on cloud-based processing for computationally intensive tasks such as computer vision and speech recognition. This choice is motivated by the limited processing capabilities, power constraints, and form-factor requirements of wearable devices.

Offloading perception and reasoning to the cloud enables the use of advanced models while keeping the wearable device lightweight and cost-effective. The primary trade-off is reliance on network connectivity, which is acknowledged as a limitation and a potential area for future exploration of edge or hybrid alternatives.

Task-Scoped Perception

A central design principle of the proposed system is task-scoped perception, in which sensing and processing are activated only in response to explicit user requests. This contrasts with continuous narration approaches that provide ongoing environmental descriptions regardless of user intent.

By limiting perception to active tasks, the system aims to reduce cognitive load, unnecessary processing, and excessive auditory feedback while preserving user control.

Wearable Form Factor with Integrated Audio

The system is designed around a wearable form factor with integrated camera, microphone, and speaker to support hands-free interaction. This design choice reflects the needs of blind users, who rely on their hands for mobility aids and tactile exploration.

Integrated audio input and output enable natural voice-based interaction without manual controls, while consistent camera placement supports stable spatial guidance.

Alternative Architectures Considered

Several alternative system architectures were considered during the design phase:

- **Smartphone-based approach:** This option was deemed less suitable due to limitations in hands-free operation and battery consumption.
- **Fully edge-based processing:** This approach was constrained by processing power, energy consumption, and form-factor limitations.
- **Hybrid edge–cloud architecture:** While flexible, this approach was deferred due to increased architectural complexity.

The selected cloud-assisted architecture represents a balanced design choice for the proposed system while preserving flexibility for future exploration of alternative processing strategies.

4.2 Hardware Setup

This section outlines the proposed hardware configuration for the Lumen wearable device. The hardware choices presented here represent a reference design intended to demonstrate feasibility and suitability for a wearable assistive navigation system, rather than a finalized or optimized implementation.

4.2.1 Hardware Components

The proposed system hardware consists of a lightweight computing platform, a camera module, audio input and output devices, and a portable power source. Table 4.1 summarizes the candidate components selected to support the system’s sensing, interaction, and communication requirements.

Table 4.1: Hardware Component Specifications

Component	Model/Type	Key Specifications
Computing Board	Raspberry Pi Zero 2 W	1GHz quad-core ARM Cortex-A53 512MB RAM 802.11n WiFi (2.4GHz)
Camera	Pi Camera Module V2	8MP, 1080p30 video, Sony IMX219 sensor
Microphone	USB Microphone	Omnidirectional audio input for voice commands
Speaker	USB Speaker	Audio output for spoken guidance
Power Supply	Portable Battery	USB-powered portable energy source

Computing Board: Raspberry Pi Zero 2 W

The Raspberry Pi Zero 2 W is selected as the computing platform for the Lumen wearable device. This single-board computer features a 1GHz quad-core ARM Cortex-A53 processor and 512MB of RAM, offering sufficient capability to support video capture, audio handling, local control logic, and network communication within a compact form factor (65mm × 30mm × 5mm).

The choice of the Pi Zero 2 W is motivated by several design considerations. Its integrated 802.11n WiFi (2.4GHz band) enables wireless communication with cloud services without additional hardware. The presence of a native Camera Serial Interface (CSI) allows direct integration with Raspberry Pi camera modules. In addition, its low power consumption and low cost make it well suited for wearable and assistive applications where size, energy efficiency, and accessibility are critical factors.

Camera Module: Pi Camera Module V2

The Pi Camera Module V2 is selected to provide visual input for object detection and environmental perception. The module is based on the Sony IMX219 8-megapixel sensor and supports video capture at up to 1080p at 30 frames per second or 720p at 60 frames per second. It interfaces directly with the Raspberry Pi Zero 2 W via a 15-pin CSI ribbon cable.

This camera module is chosen primarily for its native compatibility with the Raspberry Pi ecosystem, reducing integration complexity. Its compact dimensions (25mm × 24mm × 9mm) and low weight make it suitable for wearable deployment. The available image resolution is expected to provide adequate visual detail for indoor object recognition while maintaining reasonable data transmission requirements. The fixed-focus lens, optimized for distances beyond approximately 50cm, aligns with typical indoor navigation and object localization scenarios.

Audio Components: Microphone and Speaker

Audio interaction is supported through an external microphone for capturing user voice commands and a speaker for delivering spoken guidance. Both components are connected to the computing board via USB, providing a simple and flexible interface suitable for a wearable prototype. This configuration enables bidirectional audio interaction without introducing additional hardware complexity.

Power Supply

The wearable device is intended to be powered by a portable 5V battery source connected via USB. This approach allows flexibility in battery selection and capacity while supporting mobile, untethered operation. Treating the power source as a modular component enables future adjustments to operating time or form factor without affecting the rest of the system design.

4.2.2 Physical Design, Integration, and Mounting

The proposed hardware components are intended to be integrated into a compact enclosure suitable for wearable use. The enclosure is designed to house the computing board, camera module, and supporting interfaces while maintaining a lightweight and unobtrusive form factor appropriate for indoor navigation assistance. Emphasis is placed on forward-facing camera alignment, stable positioning during movement, and user comfort during extended wear.

Figure 4.1 illustrates the intended internal component arrangement.

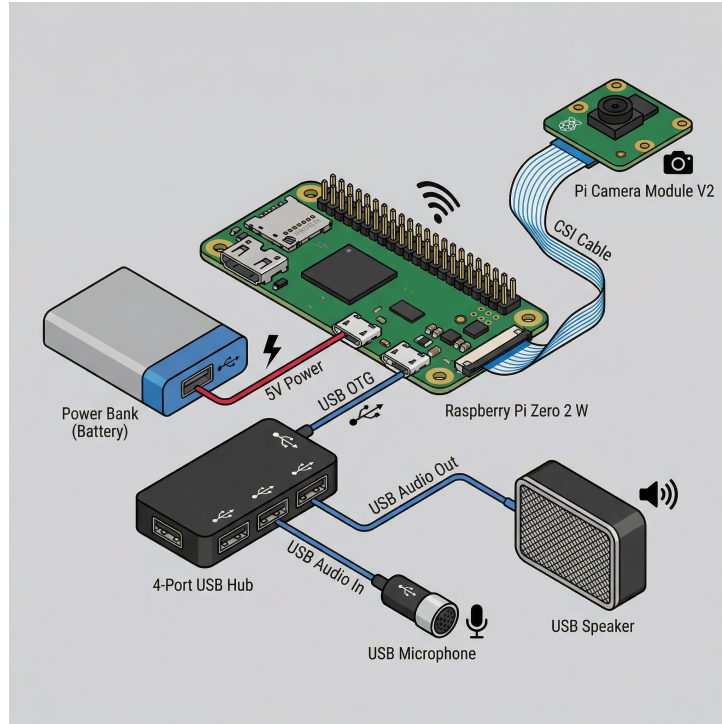


Figure 4.1: Hardware Component Diagram

For deployment, a chest-mounted configuration is considered the primary reference due to its stable viewpoint, natural alignment with user movement, and compatibility with hands-free interaction. Figure 4.2 shows the intended chest-worn configuration used as the baseline for system design.

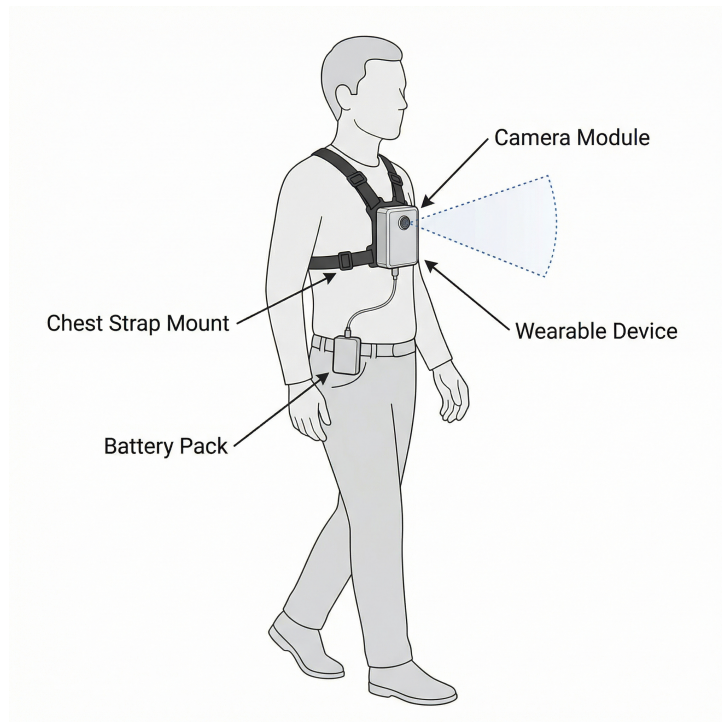


Figure 4.2: Full Mounted Device Illustration

4.2.3 System Cost

The estimated hardware cost for the proposed Lumen wearable device is approximately \$85–95 based on current market prices. This estimate includes the Raspberry Pi Zero 2 W, Pi Camera Module V2, basic USB audio components, supporting accessories, enclosure materials, and a portable battery pack. The cost is intended to demonstrate economic feasibility and accessibility rather than represent a finalized bill of materials, and may vary as the design evolves.

4.2.4 Design Rationale and Alternatives

The Raspberry Pi Zero 2 W was selected as the reference computing platform due to its compact size, low power consumption, integrated WiFi, and suitability for a cloud-based architecture. Several alternative platforms were considered:

- **Raspberry Pi 4:** Provides higher performance but at the cost of increased size, power consumption, and price, making it less suitable for wearable use.
- **ESP32-CAM:** Extremely compact and low-cost, but lacks sufficient processing capability and audio support for the proposed interaction model.
- **NVIDIA Jetson Nano:** Supports on-device AI acceleration, but its size, power requirements, and cost are unnecessary for a cloud-assisted system.

Overall, the Pi Zero 2 W represents an appropriate balance between capability, portability, and cost for the proposed system.

4.3 Software Architecture

The Lumen software architecture implements a distributed cloud-based processing model where the wearable device handles data acquisition and audio output while cloud servers perform computationally intensive AI tasks. This section describes the software components on both the device and cloud infrastructure, their interactions, and the design decisions that enable effective assistive navigation.

4.3.1 Architecture Overview

The software system is organized into two primary subsystems: device-side software running on the wearable unit and cloud-side processing services. This distributed architecture allows the system to leverage cloud computation while maintaining a lightweight wearable device.

The architecture follows a modular, service-oriented design in which each component performs a specific function. This structure supports flexibility during development and enables individual components to be modified or replaced without affecting the overall system.

Figure 4.3 presents a conceptual overview of the proposed software architecture and the interactions between major components.

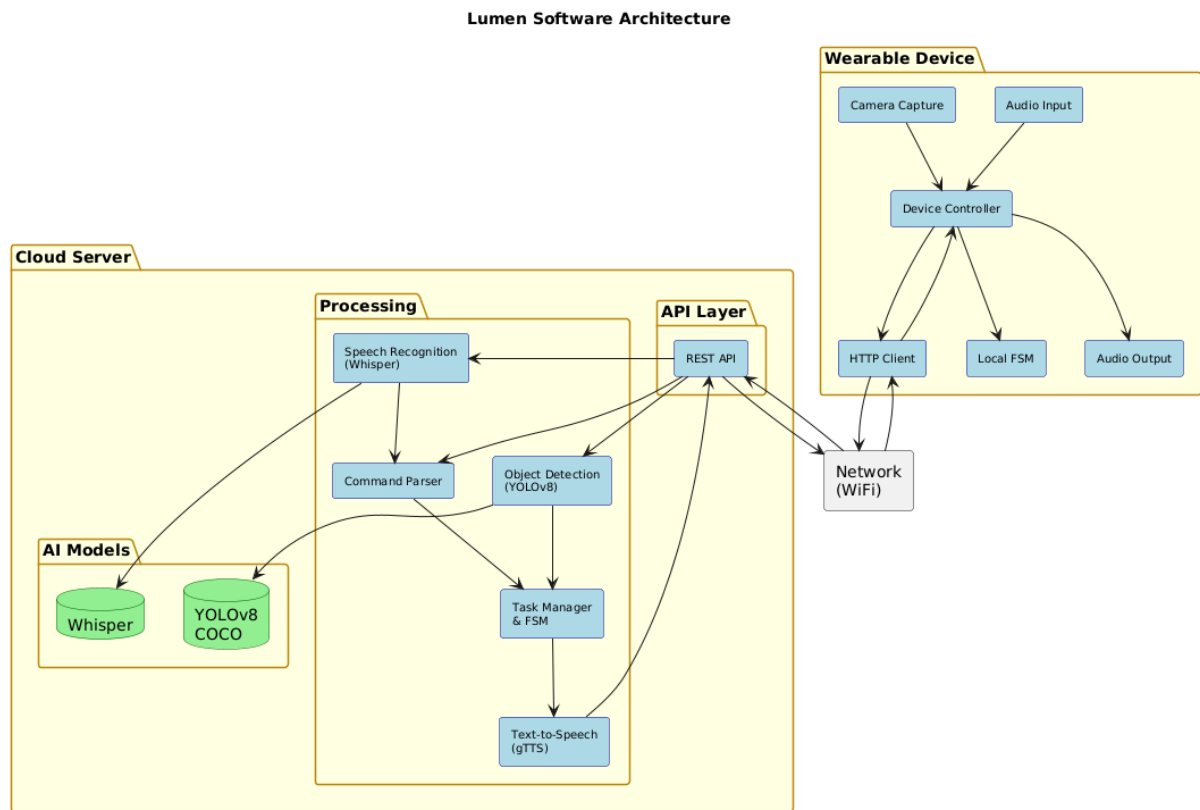


Figure 4.3: Software Architecture Diagram

4.3.2 Device-Side Software

The device-side software is responsible for interfacing with local hardware components and coordinating communication with cloud services during active tasks. It is lightweight, focusing on data acquisition and audio feedback rather than on-device intelligence.

Camera Capture Module

The camera capture module is designed to acquire visual frames from the camera during tasks and forward them to the cloud for processing. Capture parameters such as resolution and frame rate are selected to balance perceptual usefulness with network constraints.

Audio Input Handler

The audio input handler captures user voice commands and forwards them to the cloud for speech recognition and command interpretation. Audio capture is designed to be event-driven, activating only when user interaction is expected.

Audio Output Player

The audio output player receives synthesized guidance from the cloud and plays it through the device speaker. Audio playback is managed to ensure guidance messages are delivered clearly and sequentially.

Local Coordination and State Management

A lightweight local coordination layer manages when hardware modules are active and ensures consistency with the cloud-side task state. This layer tracks high-level device states (e.g., idle or task active) and ensures the device returns to a safe idle state in the event of task completion, cancellation, or communication failure.

4.3.3 Cloud Server Software

The cloud server is intended to host the core intelligence of the Lumen system, providing centralized processing for perception, language understanding, and task coordination through a REST-based service interface.

Technology Stack

The cloud services are planned to be implemented using a Python-based web framework and deployed on cloud infrastructure capable of supporting computationally intensive processing, such as deep learning inference. This setup allows the system to scale processing resources independently of the wearable device.

Speech Recognition Module (Whisper)

Speech recognition is handled by a cloud-based automatic speech recognition model capable of converting user voice commands into text. The transcribed output is used as input to downstream task interpretation and may include confidence information to support clarification when needed.

Command Parser

The command parser interprets transcribed speech to identify the intended task type and associated parameters. The system is designed to support two primary command

categories:

- **Object Allocation:** commands requesting the location of a specific object
- **Navigation:** commands requesting guidance to a destination

The parser is intended to handle natural variations in phrasing and to request clarification when a command cannot be reliably interpreted.

Dataset and Data Processing

The performance and capabilities of the object detection system are shaped by the training dataset on which the model is based. This subsection describes the baseline dataset considered for the proposed system, its relevance to indoor assistive navigation, and known limitations that inform subsequent system design decisions.

COCO Dataset Overview

The proposed object detection approach relies on model weights pre-trained on the COCO (Common Objects in Context) dataset, a large-scale benchmark widely adopted in computer vision research. COCO includes:

- **Scale:** Over 330,000 images with more than 200,000 labeled object instances.
- **Object categories:** 80 distinct classes covering common everyday objects.
- **Context richness:** An average of 7.7 object instances per image.
- **Annotation quality:** Manually annotated bounding boxes produced by multiple annotators.

Table 4.2 summarizes COCO object classes that are particularly relevant to indoor navigation and assistive tasks.

Table 4.2: COCO Dataset Classes Relevant to Indoor Navigation

Super-Category	Object Classes
Furniture	chair, couch, bed, dining table, toilet
Kitchen	bottle, cup, fork, knife, spoon, bowl
Appliance	microwave, oven, toaster, sink, refrigerator
Electronic	tv, laptop, mouse, remote, keyboard, cell phone
Indoor	book, clock, vase, scissors

These classes represent objects commonly encountered in indoor environments, including personal items, furniture that may act as obstacles or landmarks, and appliances that implicitly indicate room function (e.g., refrigerator for kitchens, toilet for bathrooms).

COCO Limitations for Assistive Navigation

Despite its broad coverage, COCO exhibits several limitations when considered in the context of indoor assistive navigation:

1. **Missing architectural classes:** Critical navigation features such as doors, windows, walls, hallways, and stairs are not included among COCO’s object categories.
2. **Viewpoint bias:** Images are predominantly captured from an eye-level perspective, whereas the proposed system assumes a chest-mounted camera with a lower and slightly tilted viewpoint.
3. **Lighting conditions:** COCO primarily contains well-lit scenes, while indoor navigation frequently occurs under variable or low-light conditions.
4. **Object scale variation:** Assistive navigation involves extreme object scales, ranging from very close obstacles to distant landmarks, which are less emphasized in COCO.

Given these constraints, the baseline system design is intended to leverage indirect landmark detection using available COCO object classes (e.g., furniture and appliances) to support initial navigation and object-finding tasks. The overall system architecture is designed to remain extensible, allowing for future dataset augmentation and model adaptation to incorporate architectural features if required.

Data Preprocessing and Label Format

Input images are expected to be resized to 640×640 pixels using letterboxing to preserve aspect ratio, with pixel values normalized to the $[0, 1]$ range. Standard data augmentation techniques—such as random horizontal flips, mosaic augmentation, random scaling and translation, and HSV color shifts—are considered to improve robustness.

COCO annotations are provided in JSON format, where each object is labeled by a bounding box $[x, y, width, height]$ in pixel coordinates along with an associated class identifier. These annotations are converted to the YOLO format $[class_id, x_{center}, y_{center}, width, height]$, with all coordinates normalized relative to image dimensions.

Object Detection Module (YOLOv8)

Based on the dataset characteristics and preprocessing pipeline described above, the proposed system adopts YOLOv8 as the object detection backbone. YOLOv8 is a state-of-the-art real-time object detection architecture that performs detection in a single forward pass, jointly predicting bounding boxes and class probabilities. This design aligns well with COCO-formatted annotations and supports efficient processing of resized 640×640 input images.

The proposed design considers the YOLOv8n (nano) variant pre-trained on the COCO dataset, inheriting support for the 80 object classes discussed in the previous subsection. The nano variant represents a favorable trade-off between detection accuracy and computational efficiency, making it appropriate for assistive guidance scenarios.

For each input frame, images are expected to be resized to 640×640 pixels before being processed by the detection network. Predictions with confidence scores below a predefined threshold are discarded. The remaining detections consist of bounding boxes, class labels, and confidence values, which are intended to be used to derive spatial information such as object center position and approximate relative distance based on bounding box size.

Task Manager and FSM

The task manager is designed to coordinate system behavior through a FSM that governs task execution flow. The FSM defines five primary operational states:

1. **Idle**: System awaiting user command
2. **ListeningForCommand**: Interpreting incoming voice command
3. **ObjectAllocationActive**: Executing an object allocation task
4. **NavigationActive**: Executing a navigation task
5. **ReturningToIdle**: Completing a task and transitioning back to idle

State transitions are intended to occur based on recognized commands, task completion conditions, timeout limits, or explicit user cancellation. During active tasks, the task manager maintains task-specific context, such as the target object for Object Allocation or the destination for Navigation.

For Object Allocation tasks, the task manager is designed to associate detected objects with the requested target, derive spatial guidance cues, and generate corresponding verbal instructions. For Navigation tasks, the manager is intended to interpret detected landmarks and produce turn-by-turn guidance.

Text-to-Speech Module

The text-to-speech module is responsible for converting generated guidance messages into spoken audio. The proposed system utilizes Google Text-to-Speech (gTTS) to synthesize natural-sounding speech, which is then transmitted to the wearable device for playback. This module enables consistent and accessible verbal feedback across all task types.

4.3.4 Data Flow and Communication Protocol

The proposed system follows a cloud-assisted design in which the wearable device handles data capture and audio output, while cloud services perform perception and interpretation. Communication is structured around discrete, task-driven exchanges rather than continuous streaming.

Visual Data Transmission

Visual data is captured and transmitted to the cloud only during active tasks. Individual frames are sent for analysis to support object detection and navigation, ensuring that visual perception remains task-scoped and inactive when the system is idle.

Audio Data Transmission

User interaction relies on discrete voice commands. Spoken input is captured on the device and sent to the cloud for interpretation, while guidance is returned as synthesized speech for local playback.

4.4 Task Definition and Execution Pipeline

The proposed Lumen system organizes assistive functionality around task-oriented interaction rather than continuous perception. Assistive behavior is structured into two primary task families—Object allocation and Navigation. This section defines these task families and describes the intended execution pipelines that coordinate perception, reasoning, and guidance in a task-scoped manner.

4.4.1 Task Family Overview

Both task families follow a common high-level execution pattern. Tasks are initiated through explicit user voice commands, activate perception only for the duration of the task, generate structured spatial guidance, and return the system to an idle state upon completion or cancellation. Although the overall interaction flow is shared, the two task families differ in their perception requirements, decision logic, and guidance strategies.

Object Allocation Tasks

Object Allocation tasks are designed to assist users in locating specific objects in their immediate environment. A task is initiated when a user issues a command such as "find my cup" or "where is the door." In response, the system is intended to activate the camera, analyze the incoming visual observations, and provide verbal directional guidance

toward the target object. The system processes each frame to detect the requested object, estimates its relative position, and generates spatial cues to help the user approach it.

These tasks are intended to support common daily activities such as locating personal belongings, identifying furniture, or finding room features. When the target object is detected at a distance, the system is designed to guide the user toward it using directional and distance cues expressed in natural language. As the user approaches, guidance is refined to reflect the changing relative position. When the object is inferred to be within pickup range, the system suggests transitioning into a dedicated hand pickup assistance mode to support the final retrieval action. An Object Allocation task is expected to conclude when the user confirms that the object has been found or picked up, the user cancels the task, or a predefined task duration is exceeded.

Navigation Tasks

Navigation tasks are designed to guide users through indoor environments toward a specified destination, such as a room or exit. Users initiate navigation by issuing commands like "navigate to the kitchen" or "take me to the exit." Rather than relying on pre-built maps, the system is designed to support reactive, landmark-based navigation using real-time perception and user-provided contextual knowledge. The system identifies decision points such as doors, hallways, and corners, and provides guidance to help users progress through structured indoor spaces.

Navigation guidance is intended to be delivered incrementally, providing turn-by-turn instructions and safety feedback while allowing users to remain in control of movement decisions. The system prompts users for waypoint descriptions, confirms when landmarks are reached, and issues obstacle warnings when hazards are detected in the path. This collaborative approach leverages the user's spatial understanding while compensating for their lack of visual feedback.

4.4.2 Object Allocation Task Execution

The Object Allocation task is designed around a structured execution pipeline that progresses from command interpretation to perception activation and guidance delivery. Figures 4.4 and 4.5 illustrate the intended interaction flow between system components during task execution.

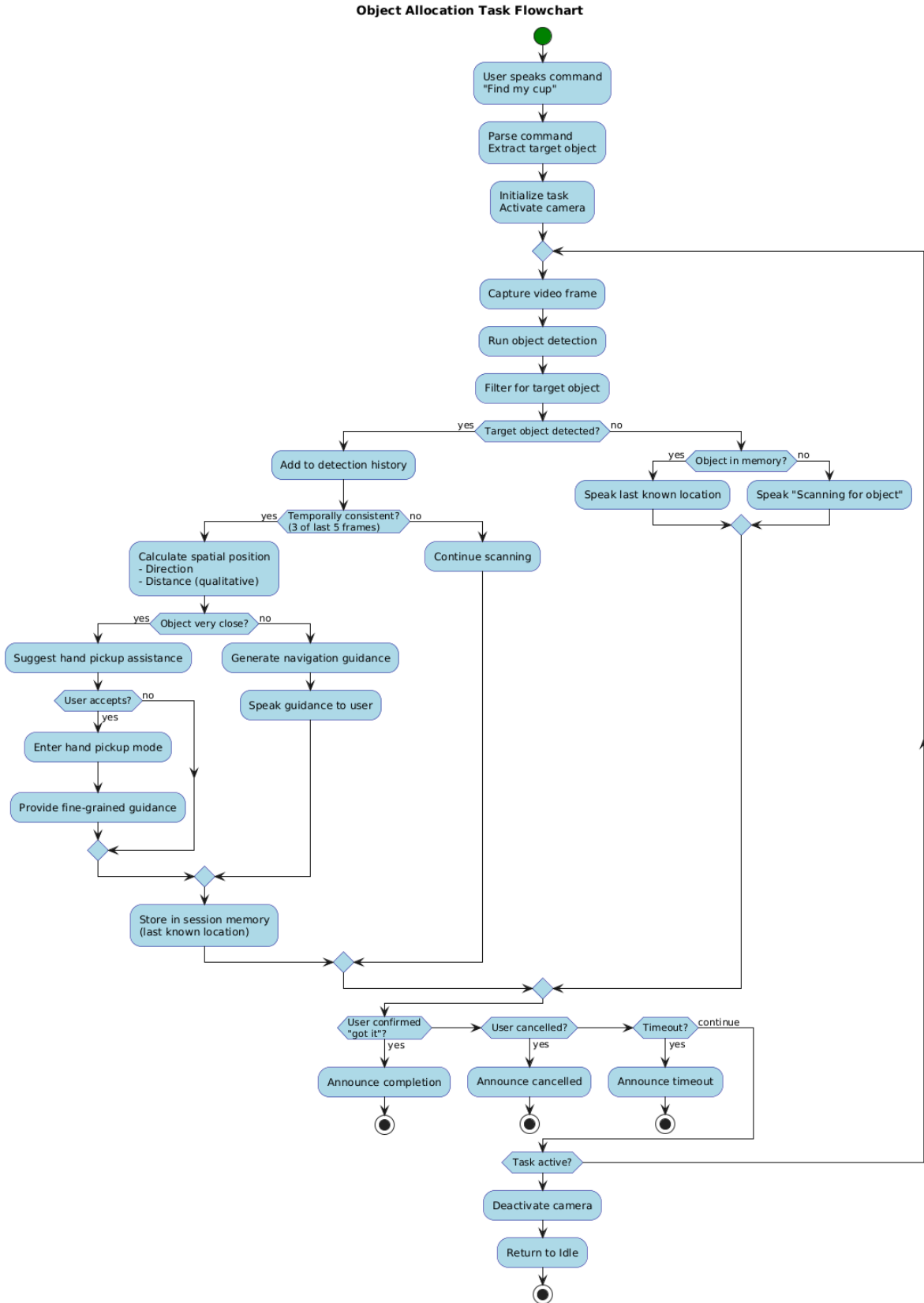


Figure 4.4: Object Allocation Task Flowchart

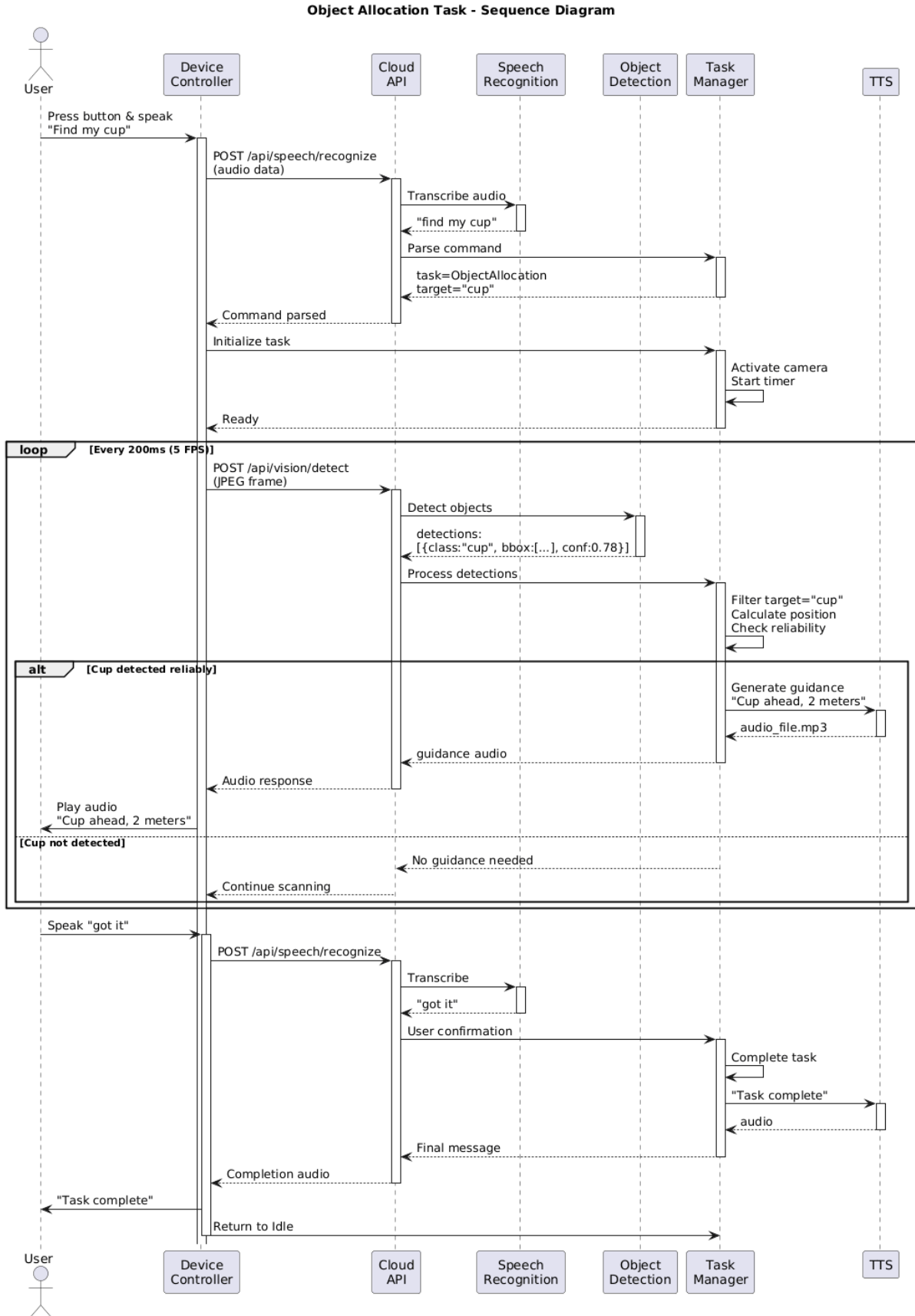


Figure 4.5: Object Allocation Task Sequence Diagram

Task Initiation

An Object Allocation task is initiated when the user issues a voice command specifying a target object, such as “find my cup” or “where is my phone.” Following speech recognition and command interpretation, the system transitions into an object-allocation active state and initializes a task context containing the target object label and any available contextual information.

Visual perception is then activated for the duration of the task, allowing the system to analyze the user’s immediate surroundings and search for candidate instances of the requested object. If the request is ambiguous or the object category is underspecified, the system may request brief clarification before proceeding. This design ensures that perception remains task-scoped and that object search is driven explicitly by user intent.

Object Detection and Temporal Consistency

During an active Object Allocation task, visual frames are analyzed to identify candidate instances of the target object specified by the user. The perception pipeline is designed to detect all visible objects in each frame and to filter these detections based on the target object class. Detection confidence and spatial metadata are retained to support subsequent reasoning and guidance generation.

To improve robustness and reduce sensitivity to transient misdetections or brief occlusions, the system design incorporates temporal consistency checks across successive observations. Rather than relying on a single frame, object presence is intended to be inferred from persistence over multiple frames within a short time window. This temporal reasoning allows the system to distinguish reliable detections from momentary artifacts, providing a more stable basis for spatial estimation and guidance.

Message Generation

Once the target object’s spatial position is estimated, the system is designed to generate natural language guidance combining object identification, directional cues, and distance estimates. The guidance generation process is intended to transform spatial parameters into natural, actionable verbal instructions delivered to the user.

Generation Approach: The proposed system employs a hybrid natural language generation approach combining template-based structures with dynamic variation. For straightforward messages, Python string templates enable rapid assembly of guidance phrases (e.g., `f"{object} detected {direction}, {distance}"`). For more natural and varied phrasing, the system may integrate an NLG library or leverage OpenAI’s API to generate contextually appropriate guidance while maintaining consistent spatial information accuracy. This hybrid approach balances computational efficiency with natural language quality.

Spatial Parameter Mapping: Directional cues are derived by dividing the camera frame into regions based on the object’s horizontal position (left, center, right), while distance estimates use qualitative categories determined by bounding box size relative to frame dimensions. This region-based approach is designed to ensure consistent directional language aligned with the user’s perspective.

Message Update Logic: To prevent overwhelming the user with continuous narration, the system is designed to implement intelligent message throttling. The guidance generator would compare the current object position to the last announced position and generate new guidance only when the object transitions between spatial regions or distance categories. This ensures updates occur only when actionable spatial information changes.

The message generation process would begin by calculating the current spatial region and distance category from the bounding box, comparing these against the last announced state. If either value has changed, direction and distance would be mapped to natural language phrases and assembled into a complete guidance message through template formatting or NLG-based generation. The message would then be passed to the gTTS API for speech synthesis and transmission to the wearable device.

Contextual Variation: The system is intended to employ brief phrase variations to avoid robotic repetition while maintaining clarity. Generated text strings would be passed to the gTTS API for synthesis into natural-sounding speech, which is then transmitted to the wearable device for playback.

Task-Specific Templates: Navigation tasks would utilize distinct message patterns compared to Object Allocation. Navigation messages are designed to focus on waypoint-based guidance (e.g., "Door ahead" or "Turn right at the corner") rather than object-distance templates, reflecting the different spatial reasoning requirements of wayfinding versus object localization.

Spatial Position Estimation

When a target object is treated as reliably detected, the system is designed to estimate its relative spatial position with respect to the user. Directional cues are derived from the object’s position within the camera frame, allowing the system to convey coarse left–right and forward–back relationships using natural language.

Distance is estimated using qualitative heuristics based on the apparent size of the detected object. Rather than producing precise metric measurements, the system groups distance into coarse, human-interpretable categories (e.g., near, medium, far). These qualitative estimates are intended to support intuitive verbal guidance and reduce cognitive load, while remaining robust to noise and viewpoint variation.

Opportunistic Object Memory

While an Object Allocation task is active, the system is designed to opportunistically cache the last known locations of selected important objects that appear within the camera’s field of view. This memory is populated only while perception is active and does not rely on continuous or background sensing.

Cached entries are time-limited and associated with explicit uncertainty. If an object is no longer visible, the system may reference its last known location using cautious phrasing (e.g., “I last saw your phone on the desk a short while ago”). This design enables limited cross-task recall while avoiding stale certainty and preserving the task-scoped perception philosophy.

Guidance Generation

Guidance during Object Allocation tasks is designed to be adaptive and context-aware. Verbal instructions are generated based on the current detection state and estimated object position, with different message types supporting initial discovery, directional correction, proximity cues, and loss of visual contact.

To avoid overwhelming the user, guidance is intended to be delivered at controlled intervals and updated only when relevant spatial information changes. This selective feedback strategy prioritizes clarity and usability over continuous narration. When the target object is inferred to be within pickup range, guidance shifts from coarse navigation cues to prompting the user to enter hand pickup assistance.

Edge Case Handling

The Object Allocation task explicitly considers the following edge cases, with priority given to cases that directly affect task success and user confidence:

- **Multiple object instances:** If multiple instances of the target object are detected, the system guides the user toward the most salient instance (e.g., closest or most centered) and may notify the user of the ambiguity.
- **Ambiguous object categories:** If the requested object corresponds to visually similar categories, the system selects the most confident match or requests clarification.
- **Partial occlusion:** If the object is partially visible, guidance continues with reduced confidence messaging.
- **Object temporarily lost:** If the object was previously detected but leaves the field of view, the system may reference its last known location and prompt reorientation.

- **Object never detected:** If the object is not detected during the task, the system prompts systematic scanning. This is treated as the primary failure case.

Hand Pickup Assistance

Hand Pickup Assistance is a dedicated interaction mode intended to support the final stage of object retrieval once a target object has been localized within close range. This mode is entered only after an Object Allocation task has guided the user sufficiently close to the object and the system infers that the object is within pickup range.

In this mode, the system is designed to provide brief, coarse-grained guidance intended to assist the user in aligning their hand with the object. Guidance focuses on relative direction and proximity (e.g., slight left/right or forward adjustments) rather than precise hand tracking or grasp planning. The system does not attempt fine-grained manipulation or autonomous grasping.

Hand Pickup Assistance is designed to be short-lived and optional. The user may confirm successful pickup, cancel assistance, or allow the system to return to idle state after completion. This separation between object localization and hand-level assistance maintains clear task boundaries while supporting safe and intuitive object retrieval.

Task Completion

An Object Allocation task is considered complete under the following conditions:

1. **User confirmation:** The user verbally confirms that the object has been found.
2. **Inferred reachability:** The object is detected directly in front of the user and appears to be within reach.
3. **User cancellation:** The user explicitly cancels the task.
4. **Prolonged non-detection:** The task expires after extended lack of progress.

Upon completion, perception is deactivated and the system returns to the idle state.

4.4.3 Navigation Task Execution

The Navigation task guides users through indoor environments using landmark-based reactive navigation. Figure 4.6 illustrates the navigation execution workflow, and Figure 4.7 shows the detailed interaction sequence.

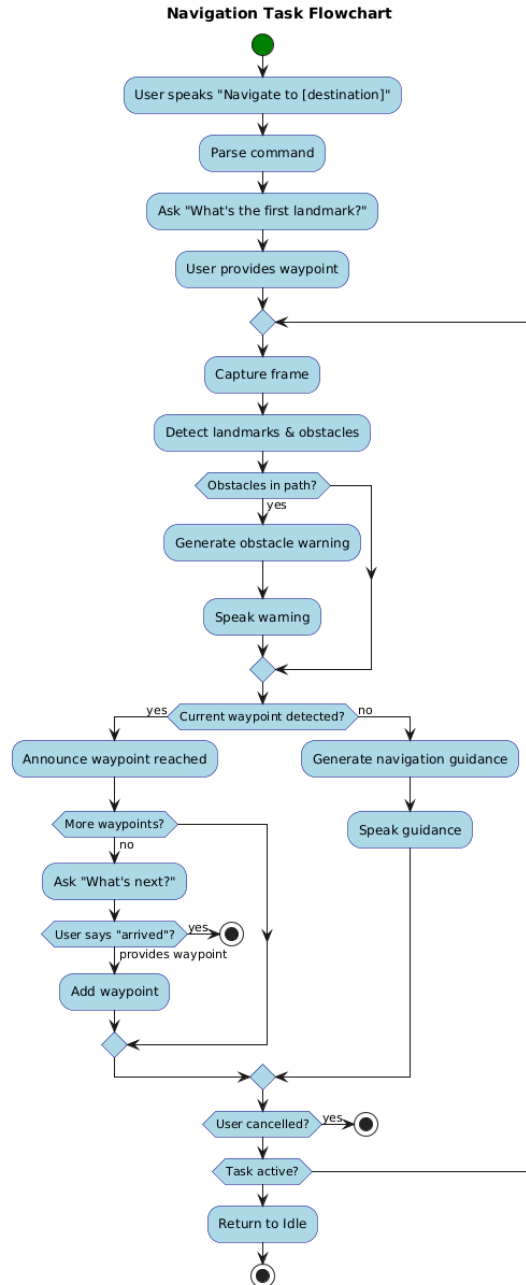


Figure 4.6: Navigation Task Flowchart

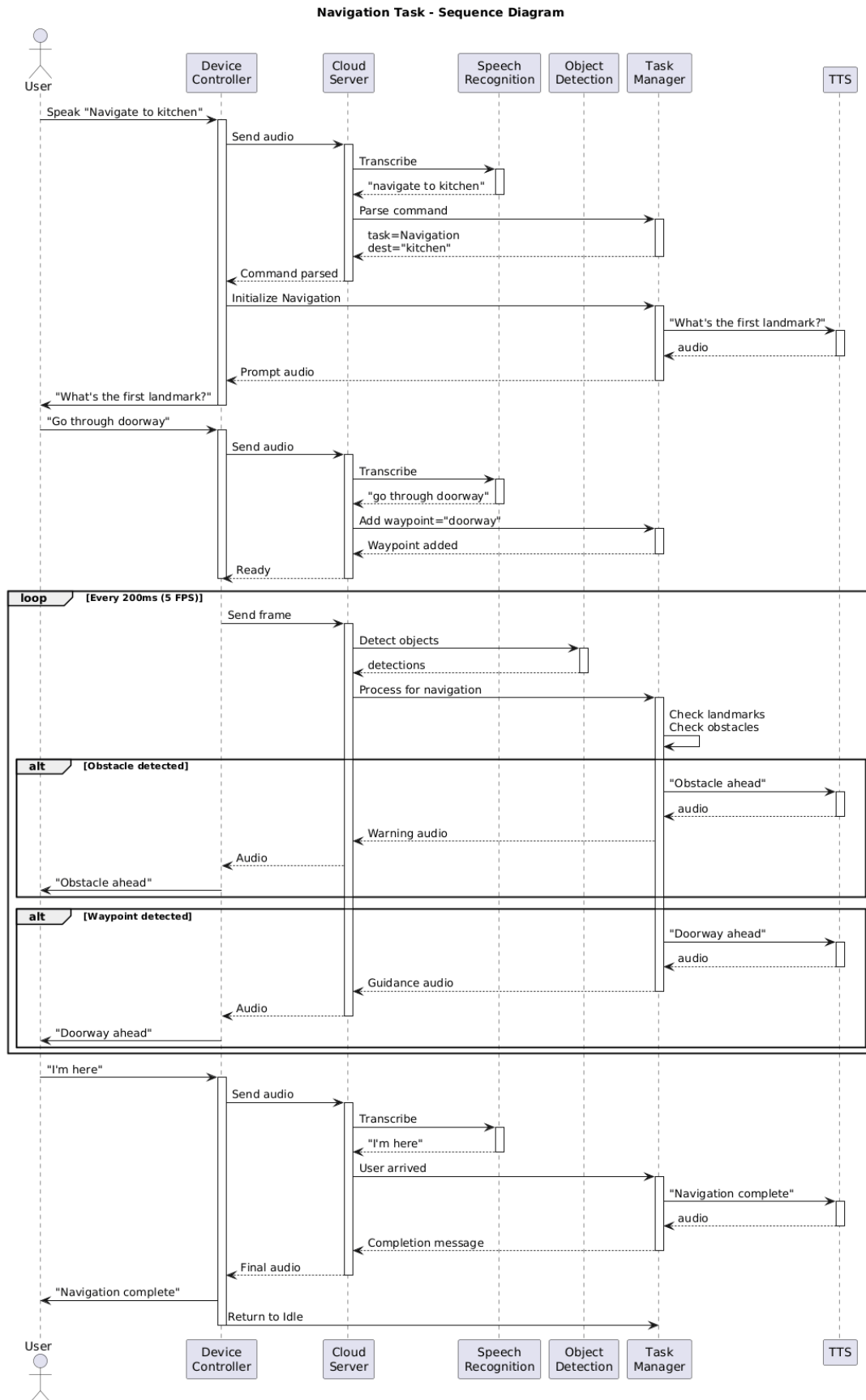


Figure 4.7: Navigation Task Sequence Diagram

Task Initiation

A Navigation task is initiated when the user issues a command such as “navigate to the kitchen” or “take me to the exit.” After speech recognition and command interpretation, the system transitions into a navigation-active state and initializes a task context containing the destination label and any available user-provided context. Visual perception is then activated for the duration of the task, enabling the system to search for navigation-relevant cues and provide structured guidance.

If the destination is underspecified or ambiguous, the system may request clarification using a short follow-up prompt (e.g., asking for the first landmark or direction). This preserves task scoping while allowing the system to operate with minimal assumptions about the environment.

Landmark-Based Navigation Strategy

The proposed navigation strategy is reactive and landmark-based rather than map-driven. Instead of relying on pre-built floor plans or global localization, the system is designed to interpret the user’s immediate surroundings and guide movement using visually detectable decision points. Landmarks may include doors, hallways, stairs, corners, and large furniture that commonly structure indoor spaces.

This approach reflects a practical assumption: users often have partial contextual knowledge about the environment (e.g., “the kitchen is down the hallway”), but lack visual feedback to confirm when they reach critical transitions or when obstacles appear. The system’s role is therefore to assist at decision points, confirm the presence or absence of expected landmarks, and provide safety feedback while the user progresses.

Waypoint Identification and User Collaboration

To support navigation without maps, the system is designed to collaborate with the user through high-level waypoint descriptions. Waypoints represent semantic milestones rather than coordinates (e.g., “go down the hallway,” “take the second door on the left,” “turn right at the corner”). When navigation begins, the system may prompt the user for the first waypoint if a route is not implicit from the destination or if additional context is needed.

As the user moves, the system attempts to confirm waypoint completion by detecting the relevant landmark and announcing it in a concise manner. Upon reaching a waypoint, the system prompts for the next waypoint if necessary. This interaction model leverages the user’s spatial understanding while providing the visual confirmation and obstacle awareness that the user cannot access directly.

Landmark Detection and Interpretation

During an active Navigation task, the perception pipeline is intended to prioritize detection of navigation-relevant landmarks. Doors are treated as critical features because they indicate transitions between rooms and represent common turning or stopping points. Hallways and open passages provide cues about forward traversal, while stairs and corners indicate major structural changes in the environment.

Detected landmarks are interpreted relative to the user’s current camera view to produce actionable guidance. For example, a door detected near the left side of the frame may be communicated as “door on your left,” while a clear corridor may be communicated as “hallway continues forward.” Landmark detection is intended to support navigation decisions rather than provide exhaustive scene description, keeping feedback task-relevant and cognitively manageable.

Turn-by-Turn Guidance Generation

Navigation guidance is generated incrementally based on the active waypoint and currently detected landmarks. Instructions are designed to be short, directional, and easy to follow (e.g., “continue forward,” “turn slightly left,” “stop here,” “door on your right”). Guidance is updated as the user progresses and as the system’s observations change.

When an expected landmark is not detected, the system is designed to respond conservatively. For instance, if the user expects a right-side doorway but none is observed, the system may inform the user that the landmark is not yet visible and suggest continuing forward or scanning. This conservative strategy prioritizes avoiding misleading instructions, especially in navigation contexts where incorrect guidance can increase user risk.

Obstacle Awareness as a Safety Layer

Obstacle awareness is designed as a safety layer that runs alongside navigation guidance. While the system provides navigational cues, it also monitors for common indoor obstacles such as chairs, tables, and people that may block the user’s path. When a nearby obstacle is detected in the user’s forward path, the system issues a brief warning intended to prompt cautious movement (e.g., “obstacle ahead” or “chair in front of you”).

Obstacle warnings are intended to take priority over navigation cues when immediate hazards are present. This ensures that safety feedback is not delayed by navigation dialogue and reinforces the system’s role as assistive rather than autonomous: the system alerts and advises, while the user controls how to proceed.

Navigation Task Completion

A Navigation task is considered complete under the following conditions:

1. **Destination reached:** The user confirms arrival (e.g., “I’m here”), or indicates the goal has been reached.
2. **User cancellation:** The user explicitly stops the task (e.g., “stop” or “cancel”).
3. **Lack of progress:** Extended inactivity suggests the task is no longer progressing and a restart or new instruction is required.

Upon completion, the system deactivates perception, clears navigation task context, provides a short confirmation message, and returns to the idle state.

4.4.4 Task Execution Examples

This subsection presents illustrative execution examples for each task type. The examples are intended to convey the intended system behavior and user experience, rather than describing a concrete implementation or measured runtime performance.

Object Allocation Example: Finding a Cup

Scenario: The user is in a kitchen and wants to locate their coffee cup.

1. **User:** Issues the command “Find my cup.”
2. **System:** Interprets the command as an Object Allocation task targeting the object category *cup* and enters an active task state.
3. **System:** Activates the camera and begins analyzing visual observations while searching for the target object.
4. **System:** After scanning the environment, identifies a cup within the user’s field of view and determines its relative position.
5. **System:** Provides directional guidance to orient the user, for example: “I see a cup on your left, across the room.”
6. **User:** Turns toward the indicated direction and moves forward.
7. **System:** Updates guidance as the user approaches the object, refining instructions based on the changing relative position (e.g., “The cup is ahead of you. Keep moving forward.”).

8. **System:** Infers that the cup is now directly in front of the user and within pickup range, and prompts the user accordingly.
9. **User:** Reaches forward and picks up the cup.
10. **System:** Confirms task completion and returns to the idle state.

Navigation Example: Going to the Kitchen

Scenario: The user is in the living room and wants to navigate to the kitchen.

1. **User:** Issues the command “Navigate to the kitchen.”
2. **System:** Interprets the command as a Navigation task and requests an initial landmark to guide the route.
3. **User:** Responds with a landmark-based instruction, such as “Go through the doorway ahead.”
4. **System:** Activates perception and searches for a doorway consistent with the user’s description.
5. **System:** Detects a doorway in front of the user and confirms it verbally.
6. **System:** Provides guidance to proceed toward the doorway.
7. **User:** Walks forward while following the guidance.
8. **System:** Detects an obstacle along the path and issues a safety warning with a suggested adjustment (e.g., “There is an obstacle ahead on your left. Move slightly to the right.”).
9. **User:** Adjusts their path and continues forward.
10. **System:** Recognizes that the doorway has been reached and prompts the user for the next navigation instruction.
11. **User:** Says, “Turn left. The kitchen is there.”
12. **System:** Confirms the instruction and provides guidance to turn left.
13. **User:** Turns and proceeds.
14. **System:** Confirms arrival and announces that navigation is complete before returning to the idle state.

These examples illustrate the intended interaction flow for both task families, demonstrating how task-scoped perception, user input, and verbal guidance are combined to support object finding and indoor navigation without relying on continuous environmental narration.

4.5 Finite State Machine Design

The operational behavior of the proposed system is structured around a FSM that governs task execution and transitions between interaction modes. The FSM provides a clear and deterministic control structure, ensuring that perception and guidance are activated only in response to explicit user requests. This design reinforces the task-scoped interaction paradigm central to Lumen and avoids continuous or autonomous system behavior.

Figure 4.8 presents the high-level FSM used to organize system operation, while Table 4.3 summarizes the intended state transitions and their triggering conditions.

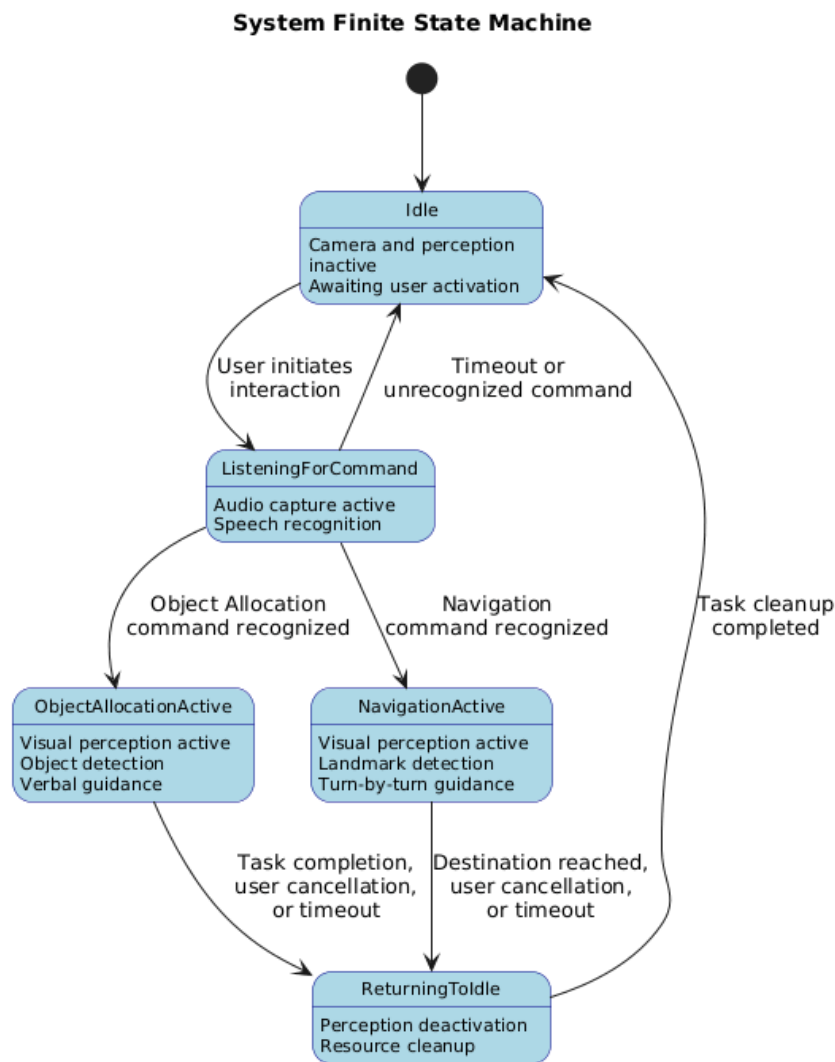


Figure 4.8: System Finite State Machine Diagram

4.5.1 FSM States

The FSM is defined using five primary states that capture the system’s interaction life-cycle:

1. **Idle**: The system remains inactive while awaiting explicit user activation. All perception modules are disabled to conserve resources.
2. **ListeningForCommand**: After activation, the system captures a voice command and transmits it for speech recognition. This state ends once a valid command is recognized or a timeout occurs.
3. **ObjectAllocationActive**: The system executes an Object Allocation task by activating visual perception and generating verbal guidance to help locate the requested object.
4. **NavigationActive**: The system executes a Navigation task, using perception to identify landmarks and obstacles and providing turn-by-turn guidance toward the destination.
5. **ReturningToIdle**: A transitional cleanup state in which perception is deactivated and the system prepares to return to Idle.

4.5.2 State Transitions

State transitions are governed by explicit events such as recognized commands, task completion conditions, user cancellation, or timeouts. Implicit or autonomous transitions are intentionally avoided. Table 4.3 summarizes the intended transition logic.

Table 4.3: FSM State Transition Rules

From State	To State	Transition Condition
Idle	ListeningForCommand	User initiates interaction
ListeningForCommand	ObjectAllocationActive	Object Allocation command recognized
ListeningForCommand	NavigationActive	Navigation command recognized
ListeningForCommand	Idle	Timeout or unrecognized command
ObjectAllocationActive	ReturningToIdle	Task completion, user cancellation, or timeout
NavigationActive	ReturningToIdle	Destination reached, user cancellation, or timeout
ReturningToIdle	Idle	Task cleanup completed

4.5.3 State Transitions

State transitions are triggered by user actions, task completion conditions, timeouts, or error events. Table 4.3 summarizes the valid transitions and their associated conditions.

The FSM is conceptually split across the device and the cloud. The device maintains a lightweight local FSM to control hardware resources such as camera activation and audio capture, while the cloud maintains the authoritative FSM that governs task execution logic.

Synchronization between the two FSMs occurs through request–response communication. If cloud communication fails or becomes unavailable, the device-side FSM is designed to transition safely back to the Idle state. This prevents unintended prolonged perception or resource usage and ensures predictable system behavior under failure conditions.

4.6 Error Handling and Recovery

The proposed system incorporates explicit error handling and recovery mechanisms to maintain safe and predictable behavior under non-ideal conditions. Rather than aiming for uninterrupted operation, the design prioritizes graceful degradation, clear user feedback, and reliable return to a safe idle state when errors occur.

4.6.1 Error Categories and Recovery Strategies

The system design considers the following primary error categories:

Network Errors: As a cloud-assisted system, Lumen depends on wireless connectivity. Potential issues include increased latency, packet loss, or connection timeouts. In such cases, the system is designed to retry requests a limited number of times and notify the user if communication fails. If connectivity cannot be restored, the system transitions to the Idle state to prevent unnecessary resource usage.

Perception Errors: Object detection and speech recognition are inherently imperfect and may produce false positives, false negatives, or low-confidence outputs. The system design mitigates these issues through confidence thresholds and temporal consistency checks for visual perception, and clarification prompts for low-confidence speech recognition results. Persistent uncertainty results in task termination rather than unreliable guidance.

User Input Errors: User commands may be ambiguous, incomplete, or unsupported. In such cases, the system is designed to request clarification or provide example phrasings. If a command cannot be resolved after repeated attempts, the system returns to Idle to avoid user frustration.

Hardware Errors: Potential hardware issues include camera, microphone, or speaker malfunction, as well as battery depletion. The device-side controller monitors the availability of expected inputs and outputs. If a hardware component becomes unresponsive, the system notifies the user and safely terminates the active task.

Task-Specific Errors: During Object Allocation, the target object may not be detected despite extended searching. During Navigation, expected landmarks may not appear or progress may stall. In such cases, the system prompts corrective actions (e.g., repositioning or restarting) and terminates the task if progress cannot be achieved.

4.6.2 Graceful Degradation

Rather than failing abruptly, the system is designed to degrade gracefully when errors occur. Under degraded conditions, perception frequency may be reduced, guidance may become more conservative, or tasks may be terminated early with clear user notification. In all cases, the system prioritizes safety, clarity, and controlled return to the Idle state over continued operation with unreliable information.

Chapter 5

Evaluation

This chapter evaluates Lumen as a task-scoped assistive system for indoor object allocation and navigation. The evaluation is designed to assess not only task success, but also the behavioral properties arising from the system’s centralized FSM, including predictability, recovery from uncertainty, and safety preemption.

5.1 Experimental Setup

Lumen is evaluated in controlled indoor environments resembling typical rooms and connecting corridors. These environments are intentionally limited in scale and complexity to reflect the system’s design assumptions: no global map construction, no long-term spatial memory, and maintenance of only the current room context. Evaluation does not assess global localization or environment reconstruction, as these are explicitly outside the scope of the proposed system.

Evaluation scenarios are constructed around the two supported task families: Object Allocation and Navigation. Tasks are initiated through voice commands, executed under the control of the centralized FSM, and terminated through successful completion, user cancellation, or safety-triggered abort. For each trial, the task type, target object or destination, and initial user position are predefined to establish task-level ground truth.

Navigation trials follow the waypoint-based collaboration model described in the methodology. Rather than evaluating global route planning, trials evaluate the system’s ability to identify semantic waypoints such as doors and corridors, confirm waypoint completion, and request clarification when destination context is insufficient.

Objects and landmarks are present in the environment; trials include viewpoint changes, partial occlusions, and cases where task-relevant targets are temporarily not detected. System logs, including perception outputs, voice recognition results, and FSM state transitions, are recorded for each trial to support quantitative and qualitative analysis.

5.2 Perception and Spatial Guidance Performance

Perception and spatial guidance performance are evaluated exclusively within the context of active task execution, reflecting Lumen’s task-scoped perception model. The system is not expected to detect or reason about objects outside the current task scope, and failure to do so is not treated as an error.

Detection success rate measures whether the task-relevant object or landmark is detected at least once during task execution when present in the environment. Relative position accuracy evaluates whether issued directional cues (e.g., left, right, ahead) correctly reflect the true spatial relationship between the user and the target. Guidance stability measures the consistency of consecutive guidance instructions during user movement, capturing the system’s ability to avoid oscillatory or contradictory commands.

For Object Allocation tasks, close-range behavior is evaluated through the Hand Pickup Assistance mode. Mode entry correctness measures whether the system transitions into pickup assistance only when the object is inferred to be within pickup range. Mode exit correctness evaluates whether the system terminates the mode through user confirmation or cancellation and returns to a valid FSM state, typically Idle.

5.3 Voice Interaction Performance

Voice interaction performance is evaluated by measuring the system’s ability to correctly recognize, interpret, and respond to spoken commands during task initiation and control. Command recognition success rate measures whether spoken commands activate the intended task type. Intent interpretation accuracy evaluates whether the correct target object or navigation destination is extracted from recognized commands.

Interaction latency is measured as the time between the end of user speech and the system’s first response, reflecting responsiveness during real-time interaction. Ambiguous or incomplete commands are not treated as outright failures; instead, clarification rate and clarification success rate measure how often the system requests additional information and how frequently such clarification loops resolve to the intended task parameters without requiring task restart.

5.4 End-to-End Task Performance

End-to-end task performance evaluates Lumen as an integrated assistive system operating under FSM control. Task completion rate measures the proportion of initiated tasks that reach successful completion according to predefined criteria. Task completion time is measured from task initiation to completion or cancellation.

Guidance efficiency is evaluated by analyzing the number of guidance instructions issued per task in relation to task success and completion time. This metric is interpreted qualitatively, as fewer instructions are not inherently better if additional guidance improves safety or user confidence.

Safety behavior is evaluated through safety intervention rate, measuring the frequency of warnings or stops issued during task execution. Correct safety stop rate measures the proportion of true hazards that trigger the intended safety response, while false safety interrupt rate captures unnecessary safety interventions in non-hazard conditions. Obstacle handling is required to preempt navigation and allocation guidance, reflecting the priority rules defined in the methodology.

5.5 FSM Behavior and Safety Evaluation

To directly evaluate the system’s core architectural contribution, FSM behavior is analyzed across all trials. Logged state transitions are examined to verify that task execution follows valid state sequences, that clarification and recovery states are entered when uncertainty arises, and that safety states correctly override normal task execution.

Each task is expected to terminate in a defined terminal state and return to Idle. The absence of illegal or undefined state transitions is treated as a necessary condition for correct system operation. Recovery-loop success rate is evaluated as the proportion of clarification or recovery activations that lead to resumed task execution or safe termination.

5.6 Qualitative Observations and Limitations

Qualitative observations are collected to complement quantitative metrics and capture behavioral properties that are difficult to measure numerically. These include perceived clarity of guidance instructions, predictability of system behavior, and user confidence during task execution. Common failure cases related to perception, speech recognition, or environmental constraints are documented.

Observed limitations are used to contextualize evaluation results and to identify directions for future work, such as expanding task scope, improving perception robustness, or supporting more complex environments. These observations reinforce the role of the FSM-based design in enabling controlled behavior, safe failure, and interpretable system operation.

Chapter 6

Conclusion and Future Work

This chapter summarizes the key contributions of the Lumen assistive navigation system, acknowledges the limitations in the current design, and outlines the planned implementation and refinement activities that will advance the system toward practical deployment.

6.1 Summary of Contributions

This work addresses the cognitive overload problem in assistive navigation systems for blind and visually impaired individuals by proposing a task-scoped, voice-controlled wearable system. Unlike existing continuous narration approaches that overwhelm users, Lumen activates perception only during explicit user-initiated tasks, maintaining a balance between proactive assistance and manageable cognitive load.

The proposed design introduces several contributions to assistive technology:

- **Task-Scoped Perception Paradigm:** A task-oriented perception paradigm that activates vision processing only during explicit user commands, eliminating continuous cognitive load while maintaining proactive assistance. This approach fundamentally differs from existing systems that provide continuous narration or require manual scanning, offering an event-driven model aligned with natural task initiation.
- **Cognitive Load Reduction Framework:** A framework for minimizing cognitive load in assistive systems through FSM-based state management and event-driven feedback. The five-state FSM ensures predictable system behavior, clear task boundaries, and graceful error handling, reducing user uncertainty and supporting independent navigation.
- **Cloud-Native Assistive Architecture:** A cloud-based architecture enabling resource-constrained wearable devices to leverage state-of-the-art computer vision

models without local processing overhead. This design separates sensing from computation, allowing the wearable client to remain lightweight and power-efficient while accessing cloud-scale AI capabilities.

- **Integrated Wearable Device with Hands-Free Interaction:** An integrated wearable design combining camera, audio I/O, and wireless connectivity with completely hands-free voice-controlled operation, requiring no manual input. The chest-mounted form factor provides stable camera positioning while preserving hand freedom for white cane use and tactile exploration.
- **Real-Time Context-Aware Guidance Pipeline:** A low-latency video streaming and audio feedback pipeline that provides spatially-aware guidance adapted to task context and user location in real-time. The system generates natural language directional cues derived from visual observations, supporting both object localization and landmark-based indoor navigation without requiring pre-built maps or environmental pre-configuration. This map-free approach enables immediate deployment in unfamiliar environments while collaborating with users’ implicit spatial knowledge.
- **Adaptive Dataset Strategy for Indoor Environments:** A two-phase dataset approach that begins with pre-trained COCO models for immediate functionality, followed by systematic custom dataset collection and model fine-tuning when architectural features (doors, stairs, hallways) are required. This strategy balances rapid prototyping with domain-specific adaptation, addressing the unique viewpoint and feature requirements of chest-mounted assistive navigation.

The proposed system demonstrates the feasibility of task-scoped assistive navigation as an alternative to continuous perception paradigms. By structuring interaction around explicit user goals, the design respects user autonomy while providing targeted visual feedback through synthesized speech.

6.2 Limitations

While the proposed Lumen system addresses key challenges in assistive navigation, several limitations inform the scope and applicability of the current design.

6.2.1 Technical Limitations

- **Dataset and Object Recognition Constraints:** The baseline object detection relies on the COCO dataset, which lacks critical architectural features such as doors,

windows, walls, stairs, and hallways. The proposed two-phase strategy mitigates but does not fully eliminate this constraint.

- **Network Dependency:** The cloud-based architecture requires reliable WiFi connectivity throughout operation. Network unavailability or connection loss will degrade or halt system functionality, limiting deployment to environments with stable wireless infrastructure.
- **Perception Latency:** The cloud-based video processing approach introduces latency that, while acceptable for the proposed object allocation and navigation tasks, may limit applicability to time-critical scenarios such as dynamic obstacle avoidance.
- **Viewpoint Mismatch:** The chest-mounted camera position differs from the eye-level perspective prevalent in COCO training data, which may reduce detection accuracy for objects at unusual angles.

6.2.2 Design Limitations

- **Reactive Navigation Only:** The proposed navigation relies on landmark-based reactive guidance rather than global path planning. While this eliminates the need for pre-built maps, it cannot provide optimal routes or warn about distant obstacles.

6.3 Future Work

The proposed Lumen system establishes a foundation for task-scoped assistive navigation, but substantial work remains to realize a deployable assistive device. This section outlines the implementation and enhancement activities planned to advance the system toward practical use.

6.3.1 System Implementation and Prototyping

The immediate priority is implementing the proposed software and hardware architecture to produce a functional prototype. Software development includes integrating the cloud server, configuring speech recognition and object detection models, implementing the task manager, and establishing reliable device-cloud communication. Hardware integration involves assembling the computing board, camera, audio components, and enclosure into a wearable form factor. Early prototyping will focus on achieving basic functionality for Object Allocation tasks before expanding to Navigation.

6.3.2 Dataset Collection and Model Refinement

If baseline COCO performance proves insufficient for indoor navigation scenarios (particularly regarding architectural feature detection) the system will be enhanced through a two-step approach. First, existing indoor scene datasets containing architectural features will be identified and evaluated for suitability. If no suitable datasets are found, custom dataset collection will be undertaken, involving image capture in representative indoor environments using the chest-mounted camera perspective, manual annotation of doors, windows, stairs, and hallways, and fine-tuning the object detection model to incorporate these architectural classes.

6.3.3 Performance Optimization

Several performance enhancements warrant investigation. Latency reduction could improve responsiveness for time-sensitive interactions. Edge processing fallback would enable operation when network connectivity is unavailable. Power optimization will be necessary to achieve practical battery life for extended use. Adaptive processing strategies may balance perception quality with resource constraints.

6.3.4 Feature Enhancement

Several enhancements could improve system utility including detailed hand pickup assistance, multi-language support, integration with existing assistive devices, and outdoor navigation extension.

6.3.5 Privacy and Security Considerations

As a cloud-based system transmitting visual and audio data, Lumen raises privacy and security concerns. Video encryption, secure storage, user consent controls, and regulatory compliance require careful implementation given the sensitive nature of captured indoor environments.

6.3.6 Implementation Timeline

Figure 6.1 presents the planned timeline for system implementation and initial validation activities over the coming months. The schedule organizes work into sequential phases, beginning with foundational software and hardware setup and progressing through task implementation, integration testing, and documentation.

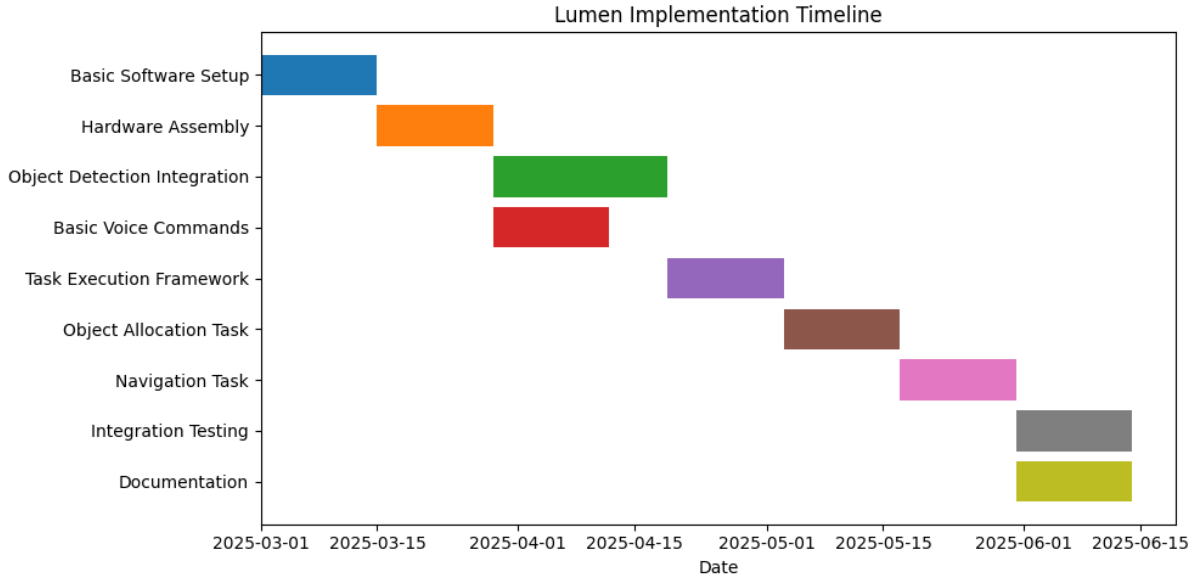


Figure 6.1: Implementation Timeline and Work Breakdown

The timeline reflects a pragmatic approach prioritizing core functionality and leaving advanced features for subsequent iterations. Flexibility is maintained to accommodate technical challenges and unforeseen complications during implementation.

6.3.7 Toward Practical Assistive Technology

The goal of this work is to contribute to assistive technology for blind and visually impaired individuals. The proposed task-scoped interaction paradigm offers a path toward systems that enhance independence without overwhelming users. By activating perception only when requested, Lumen complements existing navigation strategies. Future iterations informed by user feedback will deliver utility to the blind community.

References

- [1] R. R. A. Bourne et al., “Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment: A systematic review and meta-analysis,” *The Lancet Global Health*, vol. 5, no. 9, e888–e897, 2017. DOI: [10.1016/S2214-109X\(17\)30293-0](https://doi.org/10.1016/S2214-109X(17)30293-0). [Online]. Available: [https://www.thelancet.com/journals/langlo/article/PIIS2214-109X\(17\)30293-0](https://www.thelancet.com/journals/langlo/article/PIIS2214-109X(17)30293-0).
- [2] H. Fernandes, P. Costa, V. Filipe, H. Paredes, and J. Barroso, “A review of assistive spatial orientation and navigation technologies for the visually impaired,” *Universal Access in the Information Society*, vol. 18, no. 1, pp. 155–168, 2019. DOI: [10.1007/s10209-017-0570-8](https://doi.org/10.1007/s10209-017-0570-8). [Online]. Available: <https://link.springer.com/article/10.1007/s10209-017-0570-8>.
- [3] D. Dakopoulos and N. G. Bourbakis, “Wearable obstacle avoidance electronic travel aids for blind: A survey,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 40, no. 1, pp. 25–35, 2010. DOI: [10.1109/TSMCC.2009.2021255](https://doi.org/10.1109/TSMCC.2009.2021255). [Online]. Available: <https://ieeexplore.ieee.org/document/5205555>.
- [4] R. N. Kandalan and K. Namuduri, “Techniques for constructing indoor navigation systems for the visually impaired: A review,” *IEEE Transactions on Human-Machine Systems*, vol. 50, no. 6, pp. 492–506, 2020. DOI: [10.1109/THMS.2020.3027443](https://doi.org/10.1109/THMS.2020.3027443). [Online]. Available: <https://ieeexplore.ieee.org/document/9194356>.
- [5] B. Li et al., “Vision-based mobile indoor assistive navigation aid for blind people,” *IEEE Transactions on Mobile Computing*, vol. 18, no. 3, pp. 702–714, 2018. DOI: [10.1109/TMC.2018.2830732](https://doi.org/10.1109/TMC.2018.2830732). [Online]. Available: <https://ieeexplore.ieee.org/document/8356346>.
- [6] M. Martinez, A. Constantinescu, B. Schauerte, D. Koester, and R. Stiefelhagen, “Cognitive evaluation of haptic and audio feedback in short range navigation tasks,” in *International Conference on Computers Helping People with Special Needs*, Springer, 2014, pp. 128–135. DOI: [10.1007/978-3-319-08596-8_20](https://doi.org/10.1007/978-3-319-08596-8_20). [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-08596-8_20.

- [7] G. Hossain, A. S. Shaik, and M. Yeasin, “Cognitive load and usability analysis of r-map for people who are blind or visually impaired,” in *Proceedings of the 28th ACM International Conference on Design of Communication*, ACM, 2010, pp. 137–142. DOI: [10.1145/1878450.1878475](https://doi.org/10.1145/1878450.1878475). [Online]. Available: <https://dl.acm.org/doi/10.1145/1878450.1878475>.
- [8] A. Khan and S. Khusro, “An insight into smartphone-based assistive solutions for visually impaired and blind people,” *Universal Access in the Information Society*, 2020. DOI: [10.1007/s10209-020-00733-8](https://doi.org/10.1007/s10209-020-00733-8). [Online]. Available: <https://link.springer.com/article/10.1007/s10209-020-00733-8>.
- [9] R. Girshick, “Fast r-cnn,” *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1440–1448, 2015. DOI: [10.1109/ICCV.2015.169](https://doi.org/10.1109/ICCV.2015.169). [Online]. Available: <https://ieeexplore.ieee.org/document/7410526>.
- [10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2016, pp. 779–788. DOI: [10.1109/CVPR.2016.91](https://doi.org/10.1109/CVPR.2016.91). [Online]. Available: <https://ieeexplore.ieee.org/document/7780460>.
- [11] Neptune.ai, *15 computer vision project ideas for beginners*, <https://neptune.ai/blog/15-computer-vision-projects>, Accessed: 2026-02-06, 2021.
- [12] G. Hinton et al., “Deep neural networks for acoustic modeling in speech recognition,” *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, 2012. DOI: [10.1109/MSP.2012.2205597](https://doi.org/10.1109/MSP.2012.2205597). [Online]. Available: <https://ieeexplore.ieee.org/document/6296526>.
- [13] A. Graves, A.-r. Mohamed, and G. Hinton, “Speech recognition with deep recurrent neural networks,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, IEEE, 2013, pp. 6645–6649. DOI: [10.1109/ICASSP.2013.6638947](https://doi.org/10.1109/ICASSP.2013.6638947). [Online]. Available: <https://ieeexplore.ieee.org/document/6638947>.
- [14] Farnell, *Voice, speech and sound recognition*, <https://uk.farnell.com/technical-resources/articles/voice-speech-and-sound-recognition>, Accessed: 2026-02-06, 2023.
- [15] G. Tur and R. De Mori, “Spoken language understanding,” *IEEE Signal Processing Magazine*, vol. 28, no. 3, pp. 33–46, 2011. DOI: [10.1109/MSP.2011.940881](https://doi.org/10.1109/MSP.2011.940881). [Online]. Available: <https://ieeexplore.ieee.org/document/5756467>.
- [16] J. D. Williams, A. Raux, and M. Henderson, “The dialog state tracking challenge,” in *Proceedings of the SIGDIAL Conference*, ACL, 2013. [Online]. Available: <https://aclanthology.org/W13-4069>.

- [17] H. Zen, K. Tokuda, and A. W. Black, “Statistical parametric speech synthesis,” *Speech Communication*, vol. 51, no. 11, pp. 1039–1064, 2009. DOI: [10.1016/j.specom.2009.04.004](https://doi.org/10.1016/j.specom.2009.04.004). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016763930900056X>.
- [18] Y. Wang et al., “Tacotron: Towards end-to-end speech synthesis,” in *Proceedings of Interspeech*, ISCA, 2017. [Online]. Available: https://www.isca-speech.org/archive/interspeech_2017/wang17_interspeech.html.
- [19] L. Porzi, S. Rota Bulò, and E. Ricci, “Assistive vision systems with voice interaction for the visually impaired,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, IEEE, 2018, pp. 1–9. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018_workshops/w14/html/Porzi_Assistive_Vision_Systems_CVPR_2018_paper.html.
- [20] J. Preece, Y. Rogers, and H. Sharp, *Interaction Design: Beyond Human-Computer Interaction*, 4th ed. Wiley, 2015, ISBN: 9781119020752. [Online]. Available: <https://www.wiley.com/en-us/Interaction+Design%3A+Beyond+Human-Computer+Interaction%2C+4th+Edition-p-9781119020752>.
- [21] A. Malizia and A. Bellucci, “The artificial intelligence design process,” *Human-Computer Interaction*, 2022. DOI: [10.1080/07370024.2022.2057896](https://doi.org/10.1080/07370024.2022.2057896). [Online]. Available: <https://www.tandfonline.com/doi/full/10.1080/07370024.2022.2057896>.
- [22] L. A. Guerrero, F. Vasquez, and S. F. Ochoa, “Navigation for the blind using computer vision,” *Journal of Systems and Software*, vol. 85, no. 6, pp. 1365–1374, 2012. DOI: [10.1016/j.jss.2012.01.036](https://doi.org/10.1016/j.jss.2012.01.036). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121212000346>.
- [23] A. Rodríguez, A. Llamazares, and E. Molinos, “Accessible indoor navigation for the visually impaired using computer vision,” *Sensors*, vol. 12, no. 10, pp. 13413–13430, 2012. DOI: [10.3390/s121013413](https://doi.org/10.3390/s121013413). [Online]. Available: <https://www.mdpi.com/1424-8220/12/10/13413>.
- [24] A. Brilhault, S. Kammoun, and S. Garchery, “An object finding system for blind people,” in *International Conference on Computers Helping People with Special Needs*, Springer, 2011, pp. 564–571. DOI: [10.1007/978-3-642-21729-6_70](https://doi.org/10.1007/978-3-642-21729-6_70). [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-21729-6_70.
- [25] R. Tapu, B. Mocanu, and T. Zaharia, “Wearable assistive devices for visually impaired: A state of the art survey,” *Pattern Recognition Letters*, vol. 42, pp. 59–71, 2014. DOI: [10.1016/j.patrec.2013.10.031](https://doi.org/10.1016/j.patrec.2013.10.031). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016786551300343X>.

- [26] C. D. Wickens, “Multiple resources and mental workload,” *Human Factors*, vol. 50, no. 3, pp. 449–455, 2008. DOI: [10.1518/001872008X288394](https://doi.org/10.1518/001872008X288394). [Online]. Available: <https://journals.sagepub.com/doi/10.1518/001872008X288394>.
- [27] X. Chen, J. Yang, and Z. Zhang, “Wearable sensor-based assistive system for visually impaired people,” *Sensors*, vol. 15, no. 8, pp. 17 547–17 567, 2015. DOI: [10.3390/s150817547](https://doi.org/10.3390/s150817547). [Online]. Available: <https://www.mdpi.com/1424-8220/15/8/17547>.
- [28] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986. DOI: [10.1109/JRA.1986.1087032](https://doi.org/10.1109/JRA.1986.1087032). [Online]. Available: <https://ieeexplore.ieee.org/document/1087032>.
- [29] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018, ISBN: 9781138593736. [Online]. Available: <https://www.taylorfrancis.com/books/mono/10.1201/9780429200453>.